ATiM: Autotuning Tensor Programs for Processing-in-DRAM

Yongwon Shin*
POSTECH
Pohang, Republic of Korea
ywshin@postech.ac.kr

Dookyung Kang*
Seoul National University
Seoul, Republic of Korea
st00ne@snu.ac.kr

Hyojin Sung
Seoul National University
Seoul, Republic of Korea
hyojin.sung@snu.ac.kr

Abstract

Processing-in-DRAM (DRAM-PIM) has emerged as a promising technology for accelerating memory-intensive operations in modern applications, such as Large Language Models (LLMs). Despite its potential, current software stacks for DRAM-PIM face significant challenges, including reliance on hand-tuned libraries that hinder programmability, limited support for high-level abstractions, and the lack of systematic optimization frameworks. To address these limitations, we present ATiM, a search-based optimizing tensor compiler for UPMEM. Key features of ATiM include: (1) automated searches of the joint search space for host and kernel tensor programs, (2) PIM-aware optimizations for efficiently handling boundary conditions, and (3) improved search algorithms for the expanded search space of UPMEM systems. Our experimental results on UPMEM hardware demonstrate performance gains of up to 6.18× for various UPMEM benchmark kernels and 8.21× for GPT-J layers. To the best of our knowledge, ATiM is the first tensor compiler to provide fully automated, autotuning-integrated code generation support for a DRAM-PIM system. By bridging the gap between high-level tensor computation abstractions and low-level hardware-specific requirements, ATiM establishes a foundation for advancing DRAM-PIM programmability and enabling streamlined optimization.

ACM Reference Format:
Yongwon Shin, Dookyung Kang, and Hyojin Sung. 2025. ATiM: Autotuning Tensor Programs for Processing-in-DRAM. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25), June 21–25, 2025, Tokyo, Japan.* ACM, New York, NY, USA, 17 pages. https://doi.org/10.1145/3695053.3731096

1 Introduction

Modern applications are increasingly data-driven, leveraging vast amounts of data to optimize processes and improve efficiency. This has led to the inevitable shift of performance bottlenecks from compute unit efficiency to limited memory performance in computing system design. Among various hardware and software-based approaches to continue performance scaling beyond the "memory wall", processing-in-DRAM (DRAM-PIM) confronts the challenge by placing compute units near memory banks and computing directly on data in the memory instead of moving them. Recently, many hardware and memory vendors presented commercial and prototype DRAM-PIM products [12, 28, 36] to showcase their potential, providing up to 16× higher in-memory compute bandwidth without data transfer overheads. For example, DDR4-based UPMEM [12] includes a RISC core called DPU (Data Processing Unit) per memory bank, allowing it to fully utilize the internal memory bandwidth and bank-level parallelism for accelerating memory-intensive workloads such as database systems, genomic analysis, and machine learning/deep learning inferences [4, 6, 11, 18, 22, 37].

For DRAM-PIM to evolve into a mainstream architectural player, a fully functioning, end-to-end software stack that provides user interfaces and code generation support for programmability and multi-level optimizations for performance is crucial. However, current DRAM-PIM software stacks are still in an early and experimental phase, focusing on supporting a narrow set of target workloads with performance advantages over CPU/GPU. They provide a limited set of heavily hand-tuned libraries [31, 35] for memory-intensive, PIM-friendly operations (e.g., GEMV) based on low-level programming model and compiler support [5, 23]. Without high-level abstraction layers bridging the gap between such operations and different DRAM-PIM backends (e.g., C programming for UPMEM), substantial programming and engineering efforts must be repeated.

Recent research leveraged modern tensor-oriented intermediate representations (IRs) and compilers [34, 47, 61] to provide such abstractions. In tensor compilers, multi-dimensional arrays, i.e., tensors, are the base data type, and tensor IR instructions define operations repeated on tensor elements. Thus, tensor IRs can seamlessly represent in-memory computations as nested loops over tensors stored in the memory, facilitating lowering from high-level tensor operations to target-specific instructions. Prior work leveraged tensor IRs to generate codes for different types of PIM hardware [26] or determine feasible mappings for tensor data to memory subarrays [15], focusing on improving the programmability of the PIM software stack.

Our key insight is that the true value of tensor compilation for DRAM-PIM lies in the unique optimization opportunities provided by domain-specific tensor abstractions: specifically, *enabling effective search-based optimization, i.e., autotuning*, and *facilitating aggressive transformations that exploit high-level semantic knowledge and hardware-specific constraints during tensor IR lowering.*

First, the autotuning framework can closely interface with the DRAM-PIM code generator to search the space of host and kernel tensor programs and optimize them simultaneously. Autotuning formulates optimizations as a search problem, exploring different optimized versions of a given tensor operation to identify the best-performing candidate through hardware measurements [7, 8, 16, 67]. This approach can automatically generate codes optimized explicitly for target hardware, often outperforming expert-tuned libraries, thereby providing both programmability and performance benefits. Existing tensor compilers have shown that the autotuning frame-

*Both authors contributed equally to this research.

ISCA '25, Tokyo, Japan

ACM ISBN 979-8-4007-1261-6/25/06
https://doi.org/10.1145/3695053.3731096

work can effectively optimize tensor kernel functions on CPUs and GPUs by sampling candidates with varying loop structures and optimization parameters such as loop tiling and unroll factors [16, 67]. While CPU and GPU kernel functions can be optimized independently, UPMEM kernel optimization depends on host-code parameters as the host controls how to distribute tensors across memory banks. This interdependency creates a larger and more complex search space, making the autotuning approach an even more compelling solution for optimal code generation for DRAM-PIM.

In addition, tensor compilers can leverage target loop structures and DRAM-PIM hardware features for aggressive, correctness-guaranteed high-level optimizations. Specializing optimization passes for specific operations and hardware can enable optimizations that would be infeasible in low-level compilers without semantic context. These PIM-aware optimizations complement autotuning and low-level approaches, offering unique performance benefits.

In this paper, we present ATiM, an optimizing tensor compiler for modern DRAM-PIM. ATiM extends the autotuning framework [52] and tensor-level IR lowering and optimization support [16] in the Apache TVM compiler [7] to provide fully automated search-based code generation for UPMEM [12, 13]. ATiM expands the autotuner to sample candidates from a joint search space including host-to-DPU data distribution strategies and kernel loop-level optimization parameters. While TVM schedule primitives are originally designed for loop transformations in kernel loops, ATiM repurposes them to simultaneously optimize UPMEM host and kernel operations. It also provides the lowering passes to translate these primitives to loop-based tensor IR programs.

Tightly integrating autotuning with code generation, in turn, enables ATiM to handle the larger and more complex search space and reduce the evolutionary search overheads. It refines the evolutionary search mechanism and filters infeasible candidates early in the autotuning process by systematically leveraging UPMEM hardware constraints and parallel execution behaviors. In addition, ATiM implements tensor-level code optimizations to aggressively reduce boundary check overheads, exploiting target algorithm and hardware knowledge. Experimental results show that ATiM-compiled kernels outperformed hand-tuned UPMEM libraries [5, 23] by up to 6.18× and 8.21× for various PIM benchmark kernels and GPT-J layers, respectively, through more efficient host and kernel code generation with optimized tiling factors and reduction strategies. To the best of our knowledge, ATiM is the first effort to provide fully automated and autotuning-integrated code generation support for commercial DRAM-PIM within a streamlined tensor compilation flow. ATiM is publicly available at: https://github.com/SNU-CODElab/atim.git.

The main contributions of the paper are as follows:

- We develop a fully automated framework for search-based code generation for UPMEM systems. By reusing and extending TVM schedule primitives, ATiM defines and explores the joint search space for host and kernel optimizations. It also provides code generation passes that translate autotuned schedule primitives to executables for UPMEM.
- We identify key performance bottlenecks in UPMEM systems and implement PIM-aware optimization passes at the tensor IR level. ATiM's boundary check elimination passes leverage high-level semantic knowledge and UPMEM's architectural features for significant performance gains.
- We enhance the TVM autotuning framework to handle the expanded and more complex search space for UPMEM. ATiM refines the search mechanism to minimize sampling noises and biases early in the autotuning process, improving the final result quality.

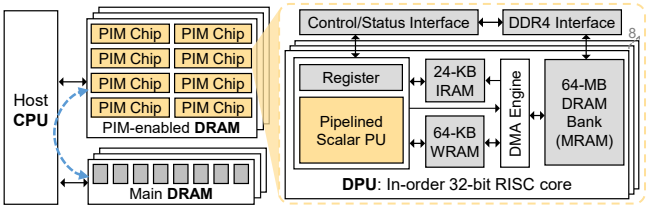


Figure 1: UPMEM architecture [12].

In the rest of the paper, we provide background information for DRAM-PIM architectures and software stacks in Section 2. Section 3 outlines motivations behind ATiM, while Sections 4 and 5 detail the design and implementation of ATiM. Section 6 and 7 present the methodology and experimental results. Section 8 discusses limitations and future directions, and Section 9 reviews related work. Finally, Section 10 concludes the paper.

2 Background

2.1 Processing-in-DRAM (DRAM-PIM)

Processing-in-memory technologies have a long history of influential research and industry proposals. While they share the ultimate goal of reducing data transfer overhead by computing where data is, they differ in processing unit (PU) design, memory types and structures, execution models, and software interfaces [9, 14, 19, 41, 42]. Among them, processing-in-DRAM or DRAM-PIM specifically targets extending DRAM architectures (including HBM) to incorporate compute units that can internally compute on data stored in the memory banks. However, it is only with recent advances in memory design and manufacturing technologies that several hardware and memory vendors have released their commercial and prototype DRAM-PIM products [12, 27, 35, 36]. While [27, 35, 36] focus on closely integrating MAC acceleration logic with the memory array so that computations can actually happen on memory access paths, Devaux [12] proposes putting a small core side by side with a data array per memory bank. This paper focuses on supporting the latter architecture with more general-purpose in-memory computing capabilities to showcase ATiM's tensor-level code generation and optimization capabilities.

UPMEM DPU. The UPMEM architecture consists of a host CPU, standard DRAM main memory, and PIM-enabled DRAM, as shown in Fig. 1. The PIM-enabled memory includes multiple chips, each with eight Data Processing Units (DPUs) capable of processing data. Since each DPU communicates directly with a single memory bank, UPMEM fully utilizes the internal bank bandwidth. Each DPU features a 32-bit RISC-style pipelined in-order core with 24 threads ("tasklets" for UPMEM), sharing a 24 KB Instruction RAM (IRAM) for instructions and a 64 KB Working RAM (WRAM) for operational data, alongside a Main RAM (MRAM) corresponding to the memory bank. Data transfers between these components are managed by a DMA engine. In a UPMEM server, all data movement between the host CPU and DPUs must pass through the host CPU;

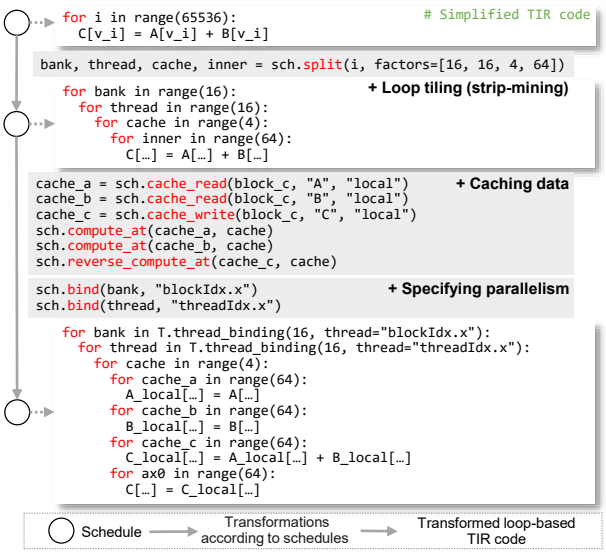


Figure 2: TIR transformations according to schedule primitives.

even when data transfer between DPUs is required, it is routed via the host CPU. Because these transfers rely on limited memory channels between the host's main memory and MRAM, reducing data movement significantly impacts performance.

DPU programming model. DPU programming is performed in an SPMD style using C, similar to OpenCL [57] or CUDA [43], and the code is compiled via LLVM [33]. Host code for DPU allocation, data management, and kernel execution can be written using the user interface provided by the UPMEM SDK. Frameworks and libraries have been developed to simplify programming on UPMEM [5, 23, 26]. PrIM [23] proposes general programming guidelines for UPMEM and provides core kernel implementations as a library. It directly uses the C language and runtime functions provided by the UPMEM SDK. SimplePIM [5] proposes an efficient UPMEM programming method with a concise interface, inspired by the similarity between UPMEM and distributed computing—both distribute data and perform independent computations on each node (DPU). It simplifies the programming process by utilizing operations and communication primitives (e.g., `map` and `reduce`).

2.2 TensorIR

TensorIR (TIR) is an intermediate representation (IR) in the TVM compiler stack [7] designed specifically to generate and optimize high-performance code for tensor computations [16]. TensorIR's main idea is to separate high-level computational definitions from the implementation details. TensorIR provides users with Python-based programming interfaces to define abstract computational tasks and let separate "schedule primitives" determine how the computation is optimized and compiled to low-level codes.

Schedule primitives. Inspired by Halide [47], TVM schedule primitives can specify flexible loop transformations without writing complex codes. Fig. 2 demonstrates how different schedule primitives shape loop structures. The `split` and `reorder` primitives handle loop tiling and reordering. While `cache_read` and `cache_write` mark data to be cached and written back, `compute_at` and `reverse_compute_at` determine its caching locations within loops. To enable parallel execution, the `bind` primitive assigns loops to threads. In Fig. 2, the outermost loop is bound to `blockIdx.x` (representing a thread block, as in GPU programming), and the immediate inner loop is bound to `threadIdx.x` (individual threads). These schedule primitives can be combined to generate codes with multiple transformations applied simultaneously.

TIR lowering process. Although there is no official distinction, schedule primitives are lowered to TIR with explicit loops, i.e., loop-based TIR, in the process of lowering TIR to target hardware. While high-level TIR representations include metadata such as block and tensor information along with schedule primitives, they are translated into an imperative language code format in loop-based TIR to facilitate low-level IR generation such as LLVM IR. A loop-based TIR program is further lowered to separate TIR programs for host and DPU kernels, which are provided as input for the UPMEM backend.

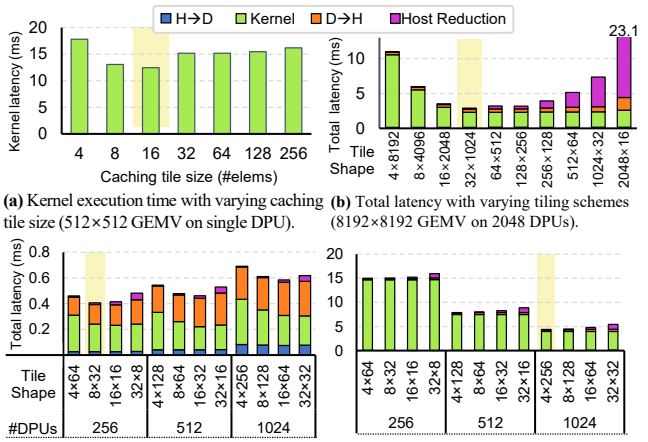
Autotuning support. Autotuning is a search-based optimization technique that explores the code space for optimal performance based on profiled runs. TVM provides autotuning support at the TensorIR level [16, 52]. TVM generates autotuning candidates in the search space by applying different schedule primitives and assigning random values to parameters. It then repeatedly measures them on hardware to find the optimal version. The searches are guided by a performance prediction model.

3 Motivating Observations

In this section, we present key observations about current software stacks and hardware behaviors of UPMEM that motivate and guide the design of ATiM.

Current software stacks for UPMEM provide only low-level programming models with limited high-level abstractions, requiring substantial development and tuning effort. Offloading computations to UPMEM involves programming tasks similar to GPU offloading with OpenCL or CUDA: writing host code to distribute data, launch kernels, and post-process results as well as PIM kernel functions for each DPU. DPU programming is even more demanding and labor-intensive than GPU programming because UPMEM lacks hardware runtime features to provide flexible resource mapping and address translation, e.g., hardware scheduler or special base address registers, which forces developers to exactly address host and kernel data. Library-based approaches [23] offer reusable building blocks to simplify programming, but host and kernel codes are still explicitly written and tuned for a specific workload, often in several hundreds of lines.

High-level abstractions such as [5, 26] can provide more extensible and general-purpose code generation support, but current solutions only provide a limited set of abstractions or fail to fully automate the process. For example, SimplePIM [5] only provides one-dimensional tensor abstractions, requiring extra effort to handle higher-dimensional data. While CINM [26] supports tensor operations such as GEMV, GEMM, and histogram with multi-level IRs to automate code generation, it still requires programmer intervention for tasks like aggregating DPU data on the host. Moreover, no existing solutions provide optimization passes that systematically transform host and kernel codes for DRAM-PIM efficiency, leaving their claims for potential tensor-level optimizations unverified. To



(a) Kernel execution time with varying caching tile size (512×512 GEMV on single DPU).

(b) Total latency with varying tiling schemes (8192×8192 GEMV on 2048 DPUs).

(c) Total latency with varying tile shapes and the number of DPUs (512×512 GEMV (left) and 8192×8192 GEMV (right)).

Figure 3: Performance impact of caching tile sizes, tiling schemes, and the number of DPUs.

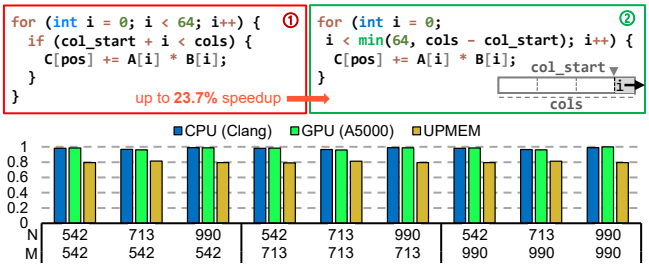


Figure 4: Boundary checks' impact on GEMV kernel (M × N) execution time.

address these gaps, we introduce ATiM, which offers flexible code generation for both host and kernel codes, enabling search-based optimizations with manual tuning.

Intra-DPU and inter-DPU optimizations have a vast search space of closely correlated parameters with significant performance impact. Preliminary experiments (Fig. 3) reveal that DPU performance is significantly impacted by *inter-DPU parallelism*, such as how data and computations are distributed across how many DPUs, as well as *intra-DPU optimizations* including loop parallelization, tiling, and caching strategies within each DPU. At the inter-DPU level, how tensor data is partitioned and distributed across DPUs ("tile size") determines kernel execution and host reduction time, with optimal tile sizes minimizing both (Fig. 3(b)). The performance impact of tile sizes also depends on the number of DPUs used relative to the original tensor's size and shape; for smaller tensors (e.g., 512×512), fewer DPUs than the maximum available can result in better performance (Fig. 3(c)). Once data is distributed, intra-DPU optimization parameters, such as tasklet parallelism, loop tiling and unroll factors, and WRAM size, are critical for DPU kernel performance (Fig. 3(a)). Optimized kernel execution time, in turn, affects inter-DPU performance trends, creating a closely correlated optimization space between the two levels. Previous work showed the benefits of heuristic-based approaches for optimizing inter-DPU data distribution [5]. In contrast, ATiM focuses on systematic autotuning, expanding the optimization space with PIM-specific parameters from both intra-DPU and inter-DPU optimizations to identify the most efficient configurations.

Table 1: Features supported by UPMEM and prior work.

	PrIM	SimplePIM	CINM	ATiM (ours)
High-level programming abstractions	✗	✓	✓	✓
High-dimensional support	✗	✗	✓	✓
Inter-DPU optimization	✗	✗	✓	✓
Intra-DPU optimization	✓	✗	✓	✓
PIM-aware optimization	✓	✓	–	✓
Autotuning support	✗	✗	✗	✓

UPMEM compute units can suffer from underutilization due to unoptimized branches. Since DPU kernels for tensor computations iteratively fetch tensor data from MRAM to WRAM at high internal DRAM bandwidth, maximizing DPU core utilization is crucial to maximize the overall compute efficiency. However, prior work observed that simple in-order DPU cores without latency-hiding hardware make the system strongly compute-bound and highly susceptible to severe underutilization from frequent branch instructions [24]. While uPIMulator [24] explored architectural improvements to mitigate branch penalties, our focus is on code-level optimizations to eliminate branches. As shown in Fig. 4, our preliminary experiments show that eliminating a redundant boundary check can provide up to a 23.7% speedup for GEMV operations. The caveat is that such optimizations are not always applied automatically by low-level compilers like LLVM, which lack high-level loop and dependency information. Furthermore, this performance impact is particularly prominent for PIM architectures, where limited compute resources amplify the benefits of software-level branch optimizations. As shown in Fig. 4, CPUs and GPUs see minimal performance gains, likely due to advanced hardware branch handling and latency-hiding mechanisms, while UPMEM achieves an average 20% runtime reduction across different input sizes. This highlights the need for aggressive optimizations at the tensor level to specifically optimize for DPU core utilization.

To address these challenges, ATiM provides an extensible, optimizing tensor compiler for UPMEM. As shown in Table 1, ATiM supports fully automated code generation for a broad range of operations with diverse tensor shapes and dimensions, while enabling autotuning in the intra-DPU and inter-DPU optimization space – capabilities that are not comprehensively supported by prior work.

4 System Overview

Fig. 5 illustrates the process of generating optimized UPMEM host and kernel codes from high-level operations through an autotuning framework and PIM-aware tensor-level transformations. The autotuner (green box) and the code generator (gray box) are technically separate components, since the code generator can also take arbitrary tensor programs as input, including hand-written schedule primitives. However, ATiM assumes the autotuner always initiates the code generation process (① to ④), except for the final compilation step when the code generator directly compiles host and kernel binaries using autotuning results (⑤).

When given a target operation to offload, the autotuner generates schedule primitive sequences using the sketch generation rules and sampled optimization parameters to populate the space of possible host and kernel code implementations. Once these rules generate schedule primitives to determine code structures, tunable parameter values are sampled within loop bounds. Then, ATiM performs an evolutionary search guided by a cost model to identify promising candidates to compile and evaluate on hardware. ATiM extends the

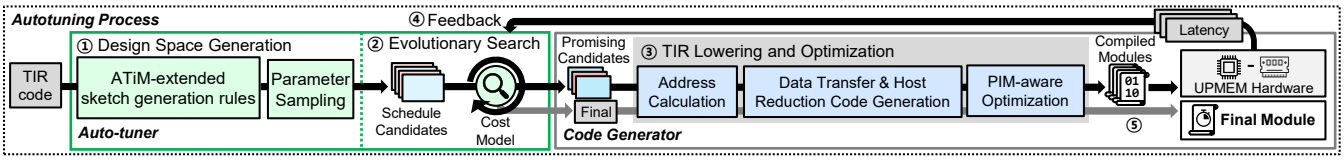


Figure 5: The overall system overview of ATiM.

existing TIR lowering passes in TVM to translate schedule primitives to represent correct tensor programs for UPMEM host and kernel operations. ATiM implements address calculation logic for kernel memory accesses to tiled data per bank, and added PIM-specific TIR lowering passes for kernel and host reduction loops and host data transfer function calls. ATiM also applies PIM-aware optimizations that leverage high-level semantic knowledge and PIM hardware constraints. The resulting host and kernel programs are compiled by the TVM backend using the LLVM compiler and TVM runtime, and evaluated by the autotuner on UPMEM hardware to collect performance results and train the cost model to guide the search.

5 ATiM: Design and Implementation

5.1 Tunable Host and Kernel Operations

Offloading tensor computations to UPMEM involves a sequence of host and kernel operations that, at a high level, follow a standardized process. We first outline these actions at an abstract, algorithmic level without specifying implementation details, so that we can separate out detailed "schedules" and optimize them through targeted searches [16].

Host code determines how to tile input and output tensor data across DPUs and aggregate results. This is a global optimization problem with intractable complexity, with conflicting trade-offs between intra- and inter-DPU parallelism and communication/synchronization overheads. ATiM addresses this by identifying key tunable parameters and applying search-based optimizations to generate efficient host code. Tiling strategies, parallelism levels, and host-to-DPU data transfer granularity impact both kernel performance and host-DPU communication cost, while thread-level parallelism can accelerate the post-processing on the host.

On the other hand, autotuning DPU kernel functions focuses mainly on optimizing loop structures, much like GPU kernel autotuning. Loop structures define which loops are parallelized (spatial loops) and which cache intermediate results (reduction loops). Unrolling and multi-level tiling factors affect tasklet-level parallelism and WRAM locality, while MRAM-WRAM caching locations and sizes decide intra-DPU data transfer overheads. Reduction strategies affect both host and kernel code operations, depending on whether results are partially reduced on DPUs first then aggregated on the host, or sent directly to the host.

In the following sections, we describe how we extend the existing tensor compiler to support tunable code structures and parameters for UPMEM, search for optimal schedules, and lower them to host and kernel implementations.

5.2 Search-based Code Generation

5.2.1 *Schedule Candidate Generation.*

ATiM extends the sketch generation rules [16] in the TVM autotuning framework to generate schedule primitive samples that implement essential host and kernel operations with varying code structures and optimization parameters (Fig. 6). The core idea behind ATiM's schedule candidate generation is *repurposing TVM schedule primitives for UPMEM host and kernel code generation.* While preserving much of the original semantics of these schedule primitives, ATiM leverages them specifically to define and populate the search space for UPMEM optimizations, as shown in Table 2.

Host-to-DPU data distribution. ATiM tiles tensor data and maps them across DPUs by using `split` and `reorder` schedule primitive and annotating tiled loop variables to bind them with DPUs. Unlike prior approaches [21] that introduced fixed data distribution strategies with separate schedule primitives, ATiM enables flexible data mapping and distribution by leveraging existing loop schedule primitives. `split` and `reorder` schedule primitives are used to define the loop tiling pattern, i.e., tile sizes in each dimension and the ordering between them. In the example, `split` primitives tile the loop to iterate over `XB×YB` tiles, and `reorder` primitive specifies the row-major traversal order. Schedule candidates also perform "DPU binding" through `bind` primitive to specify which loop level corresponds to inter-DPU parallelism, i.e., determine tiles per DPU. The bank index uses linearized values of the loop variables with the annotation. In the example, the inner tiles are distributed to `XB×YB` DPUs by binding the loop variables `x_dpu` and `y_dpu`.

Reduction strategy. When multidimensional tiles are mapped to DPUs, computation results can be partially reduced in one dimension, and intermediate results can be aggregated on the host. ATiM uses `rfactor` schedule primitives to represent such parallel reduction strategies on UPMEM.

Multi-level tiling. ATiM generates a sequence of `split`, `reorder`, and `bind` primitives to define how to execute the kernel loop at each DPU. For example, `split` and `reorder` perform multi-level tiling by creating an outermost loop for multi-tasklet execution (`thread_dpu`), additional loop levels to cache data (`x_cache` and `y_cache`), and innermost loops (`y_in` and `x_in`). Then, `bind` primitives associate the outermost loop iterations with parallel thread IDs to exploit intra-DPU thread parallelism ("tasklet binding").

Intra-DPU caching. DPU kernels must explicitly load and store data in WRAM before and after computation. MRAM-WRAM caching locations and granularities (i.e., caching data sizes) significantly impact data locality and transfer overheads; per-thread caching may be too coarse-grained for WRAM, limiting parallelism, while caching inside the innermost tiling loop can trigger excessive MRAM-DRAM data transfers. ATiM allows the autotuner to search for optimal caching locations and granularities by using `cache_read/write` and `(reverse)_compute_at` schedule primitives. `cache_read/write` primitives define WRAM tiles to load or store, and `(reverse)_compute_at` binds these caching tiles with loops to specify exact caching locations.

Post-processing. After DPU kernel execution, the host code aggregates computation results from DPUs to produce the final output.

Table 2: Tunable host and kernel operations with corresponding schedule primitives.

	Schedule primitives	Tunable parameters	Codegen target	Examples
Host-to-DPU data distribution	split	tiling factor	host	`x_dpu, x_in_dpu = sch.split(x, factors=[XB, None])` `y_dpu, y_in_dpu = sch.split(y, factors=[YB, None])`
	reorder	-	host	`sch.reorder(x_dpu, y_dpu, x_in_dpu, y_in_dpu)`
	bind	-	host	`sch.bind(x_dpu, "blockIdx.x")` `sch.bind(x_dpu, "blockIdx.y")`
Reduction strategy	rfactor	-	host, kernel	`block_dpu = sch.rfactor(x_dpu, factor_axis=0)`
Multi-level tiling	split	tiling factor	kernel	`thread_dpu, y_cache, y_in = sch.split(y_in_dpu, factors=[...])` `x_cache, x_in = sch.split(x_in_dpu, factors=[...])`
	reorder	-	kernel	`sch.reorder(thread_dpu, y_cache, y_in, x_cache, x_in)`
	bind	-	kernel	`sch.bind(thread_dpu, "threadIdx.x")`

Intra-DPU caching	cache_read/write	-	kernel	`cache_a = sch.cache_read(block_dpu, 0, "local")`
				`cache_b = sch.cache_read(block_dpu, 1, "local")`
				`cache_c = sch.cache_write(block_dpu, 0, "local")`

	(reverse_)compute_at	caching location	kernel	`sch.compute_at(cache_a, x_cache)`
				`sch.compute_at(cache_b, x_cache)`
				`sch.reverse_compute_at(cache_c, y_cache)`

Post-processing	split	tiling factor	host	`thread, _ = sch.split(y, factors=[...])`
	parallel	-	host	`sch.parallel(thread)`

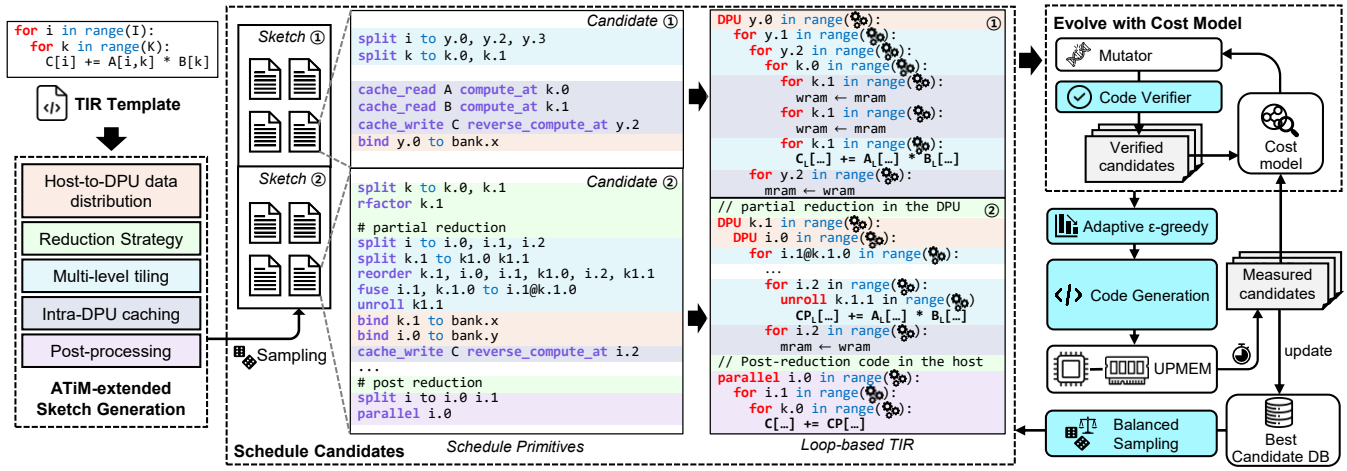


Figure 6: Autotuning-driven code generation process in ATiM.

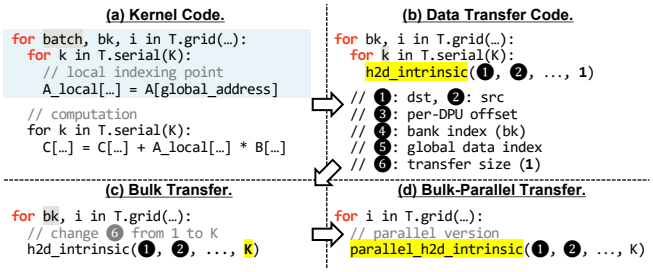


Figure 7: Data transfer code generation and optimization.

This loop can incur significant overhead if executed sequentially for a larger number of DPU results. ATiM enables tiling and parallel optimizations for this host post-processing loop by using `split` and `parallel` schedule primitives.

5.2.2 *TIR Lowering.*

The code generator for UPMEM lowers schedule primitive sequences selected from the evolutionary search to loop-based TIR programs. While ATiM reuses the current TIR lowering passes to construct nested loops, apply loop transformations such as unrolling and tiling, and generate parallel DPU kernels and tasklets per DPU, it performs the following ATiM-specific passes to generate per-DPU address calculation (kernel), data transfer function calls (host), and partial and final reduction loops (host and kernel). Final TIR programs are compiled to host and kernel binaries by the Clang/LLVM compiler in the UPMEM SDK and TVM runtime.

Address calculation. Address calculation also leverages schedule primitives to introduce a local DPU scope in compile time. Without runtime translation support such as base address registers, calculating a per-DPU offset from a global address is tedious and can incur high overheads for complicated tiling patterns, e.g., high-dimensional tiles or tiles with strides. By allocating caching tiles for WRAM using `cache_read/write` and specifying their locations using `compute_at`, ATiM allows DPU-local data on WRAM to be accessed simply by adding outer-loop indices multiplied by the caching tile size (base address) and the inner-loop indices (per-DPU offset). Since tiled data in MRAM maintains its host address, ATiM can quickly establish the mapping from the global tensor address to the per-DPU local address.

Data transfer code generation. ATiM reuses the global-to-local address mapping established in kernel codes to generate data transfers. As shown in Fig. 7, starting from the kernel TIR code generated for UPMEM (a), we adapt the loop nest to perform iterative but non-redundant data transfers by removing the outer loop `batch`, keeping only the loops outside the local indexing point (loops `bk`, `i`, and `k`), and putting a ATiM data transfer intrinsic at the WRAM caching location (b). For host-to-device transfers, this intrinsic takes the global data index as a source and the DPU index and per-DPU offset as a destination. Device-to-host transfer codes can be generated similarly, with the transfer direction reversed.

While the code in Fig. 7(b) works correctly, its performance can be improved by allowing parallel and bulk transfers. We implement

the following two optimization passes to generate more efficient data transfer codes for UPMEM (as shown in Fig. 7(c) and (d)).

- *Bulk transfer*: ATiM coalesces individual data transfers to a single transfer of a contiguous data chunk. This optimization pass visits loops with data transfer intrinsics, checks if data accesses are contiguous within a loop nest, and generates "vectorized" transfers by unrolling the loops. This process is repeated until further unrolling is impossible. As shown in Fig. 7(c), loop `k` is unrolled to increase the data transfer size from 1 to `K`, `k`'s loop extent.
- *Bank-parallel transfer*: For UPMEM with parallel data transfer support, ATiM provides parallel data transfer intrinsics. Parallel transfers of data banks (DPU) in a rank are supported by `dpu_prepare_xfer` and `dpu_push_xfer` functions, to which parallel data transfer intrinsics in Fig. 7(d) are later lowered.

Reduction code generation. The `rfactor` primitive is originally for securing additional parallelism in the reduction dimension in kernel codes. ATiM code generator translates it to hierarchical reduction with parallel partial reduction loops across DPUs and a final reduction loop on the host. The resulting loops reduce the amount of input data fetched by each DPU while generating another reduction loop over intermediate output vectors. The cost of executing the additional final reduction loop on the host CPU is quickly offset by reduced data transfers between the host and DPUs. The final reduction loop can be further tiled and executed in parallel on multi-threaded CPUs.

5.2.3 *Balanced Evolutionary Search.*

ATiM augments the evolutionary search mechanism in the current auto-tuner to explore the expanded search space more effectively, as shown in Fig. 6 (right). While autotuning for CPU or GPU focuses solely on the search space from computational subgraphs, ATiM's search space additionally includes tunable host operations, especially host-to-DPU data distribution, which determines inter-DPU parallelism. As a result, the evolutionary search for ATiM must sample schedule candidates to simultaneously optimize inter-DPU parallelism across DPUs and intra-DPU (tasklet) parallelism per DPU kernel.

We observed that this more extensive and complex search space biases the search toward inter-DPU parallelism over intra-DPU parallelism, with orders of magnitude more DPUs than per-DPU tasklets in typical UPMEM systems. Early in the autotuning process, this bias leads to overly favoring schedule primitives with `rfactor` that exploit inter-DPU parallelism for hierarchical reduction while prematurely dropping out candidates without `rfactor` from the search space, resulting in suboptimal performance.

To address this `rfactor` primitive bias, ATiM employs two techniques: *balanced sampling* and an *adaptive epsilon-greedy strategy*. The balanced sampler ensures equal representation of candidates with and without `rfactor` during early autotuning iterations. Specifically, it selects an equal proportion of top-K candidates for both design spaces (① and ② in Fig. 6) from the best candidate database. This technique promotes exploration and preserves non-`rfactor` candidates but is only applied during the first 40% of total trials to balance exploration and exploitation. After this point, top-K candidates are extracted without distinguishing between design spaces to accelerate convergence. Candidates directly sampled from the design spaces follow uniform distribution across design spaces, hence they do not require the balanced sampler.

The adaptive epsilon-greedy strategy increases the epsilon value during early autotuning phases, allowing diverse schedule candidates to be added to the best candidate database. However, maintaining a high epsilon value indefinitely can hinder convergence. Therefore, ATiM starts with an epsilon value of 0.5 and linearly decreases it to the default value of 0.05 after the first 40% of trials.

5.2.4 *Code verifier for UPMEM.*

ATiM includes a schedule verifier to filter out candidates that violate UPMEM constraints during evolutionary search. Compared to CPUs and GPUs, UPMEM imposes much stricter hardware and programming constraints. Unlike GPUs, which manage thread blocks via hardware schedulers, UPMEM requires explicit management of up to 2560 DPUs and 24 tasklets per DPU, with a 64 KB WRAM limit for caching. This often leads to invalid candidates that waste resources and degrade autotuning efficiency during an evolutionary search if unchecked. By eliminating such candidates early, the verifier reduces overhead and ensures high-quality results.

5.3 PIM-aware Optimizations

ATiM introduces tensor-level optimizations that leverage target loop structures and PIM hardware features to streamline control flows and improve PIM utilization. During kernel code generation, the TIR lowering passes generate loop-based TIR for tiled tensors and insert conditional statements to ensure that DPU tasklets access only valid memory within tensor boundaries. These boundary checks do not affect interior tiles, but they prevent memory overflows or unnecessary computations for tiles at dimension boundaries whose extents may not align with the tile size.

While boundary checks do not significantly impact performance on traditional processors thanks to latency-hiding techniques (e.g., branch predictors, ILP/TLP support), they can cause severe front-end stalls for simpler in-order DPU cores, which cannot mitigate control hazards without branch predictors and reorder buffers [24]. Strict area and power constraints on DRAM chips would not easily allow such complicated logic within a DPU. As a result, boundary checks can have a lasting, damaging impact on DPU kernel performance.

Software-level optimizations to eliminate or reduce boundary checks can be essential to minimize branch overheads in this case. While low-level compilers like LLVM can simplify control flow (e.g., merging, eliminating, or hoisting branches), most boundary checks remain unsafe to optimize at this stage, except for loop invariant code motion (LICM) partially applied to boundary equations, without high-level semantic knowledge. On the other hand, algorithm-level optimizations can enable more aggressive code rewriting by specifically targeting boundary checks. DietCode [66] proposed two optimization techniques: (1) padding data locally at tile boundary or globally per tensor data to ensure out-of-bound accesses are safe, thereby removing the need for boundary checks, and (2) partitioning the loop body into slow boundary and fast non-boundary sections [54]. While TVM implements loop partitioning, it provides limited benefits for DPU kernels, as it is effective only for large tensor shapes. Data padding can eliminate boundary checks but significantly increase storage and transfer overheads, particularly with global padding.

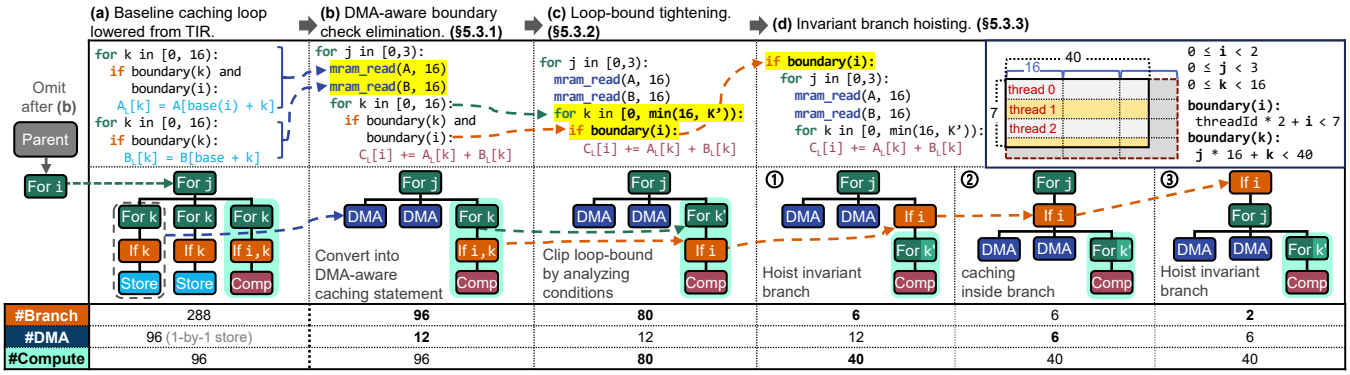


Figure 8: Example of PIM-aware optimizations applied to a 7×40 GEMV kernel (single DPU, 4 threads with 4-byte data type). The kernel processes two rows at a time to meet the 8-byte alignment for the output and caches 16 elements per iteration, forming a 2×16 tiling pattern, introducing boundary conditions along the row and column axes. The table shows the number of branches, DMA transfers, and innermost loop executions after each optimization step.

ATiM adopts the padding techniques [66] and introduces three TIR-level transformation passes that collaboratively reduce boundary checks and optimize memory accesses guarded by them using DPU functions. These passes target loop-based TIR kernel codes with affine access patterns and static tensor shapes, generated in the TIR lowering process, which are main optimization targets for tensor compilers and DRAM-PIM. These loops are not part of the autotuning targets, as the current TVM autotuner performs loop tiling in a way that prevents misaligned tiles. Since the TIR semantics for `compute_at` and `reverse_compute_at` schedule primitives enforce that all consumer operations of a loop are under the given loop's constraints [16], we can assume that target loops have no other statements outside of the boundary condition. This specific loop structure enables aggressive loop transformation and branch elimination, which low-level compilers cannot reliably validate.

5.3.1 *DMA-aware boundary check elimination.*

This optimization pass eliminates redundant boundary checks for load and store instructions for WRAM. These instructions fetch operands to WRAM from MRAM before computation and write them back to the memory bank afterward. Once boundary checks are removed, the enclosing loops become vectorizable, so we can replace each loop with a DMA instruction between WRAM and the memory bank. DMA instructions can significantly speed up data transfers for contiguous data, but they can only be used without condition statements. As shown in Fig. 8(b), this pass can replace a loop with contiguous store instructions with a DMA instruction after eliminating the boundary check.

We can safely remove these boundary checks because (1) memory regions on DPU MRAM are locally padded, i.e., allocated in multiples of tile sizes, preventing out-of-bound accesses between MRAM and WRAM from corrupting meaningful data and (2) the same boundary checks are repeated later, and as long as we keep the ones guarding the computation in the kernel and readout in the host, we ensure that host data remains protected. While this may slightly increase data transfer traffic outside the boundary, the performance gains far outweigh its minimal impact.

5.3.2 *Loop-bound tightening.*

When a boundary check condition right after the loop header is a linear inequality and there are no other instructions in the loop body, which are assumed results of the current TIR lowering mechanism, this condition can intersect with the loop's upper bound condition by solving the two linear inequalities. As illustrated in Fig. 8(c), this process tightens the upper bound of the loop variable `k` after being merged with the following boundary condition (`if boundary(k) and boundary(i)`). Foregoing "dead" iterations known to fail the boundary check in compile time can significantly reduce the loop execution time (from 96 to 80 iterations). The computation complexity of the loop-bound evaluation may increase, which can later be optimized through strength reduction. General-purpose compilers lack equivalent loop optimization passes, while previous research on polyhedral compilation exploited user-provided assumptions [20] or directives [30] to enable broader and more advanced loop optimizations. In contrast, ATiM can safely apply loop-bound tightening without requiring additional safety checks or user input by focusing on lowered IRs with structural properties guaranteed by the IR lowering process, rather than arbitrary user-written code. Once the optimization is applied, only a boundary check with a loop invariant variable (`i`) remains, which can be further optimized by invariant branch hoisting in Section 5.3.3.

5.3.3 *Invariant branch hoisting.*

Invariant branch hoisting moves boundary checks with loop-invariant conditions, which remain after loop-bound tightening, outside the loop `k` (step ①), as shown in Fig. 8(d). Invariant branches can also be hoisted out of the loop later by a low-level compiler in the "loop unswitching" optimization pass [39]. However, ATiM's invariant branch hoisting goes further by integrating loop unswitching with partial dead code elimination (PDCE) [29] to create additional opportunities for hoisting invariant branches further up. In this context, data cached by the DMA operations, such as `A` and `B` in Fig. 8(d), become partially dead outside the `boundary(i)` condition, since the TIR lowering pass enforces that all the consumers of the loop are under the loop's constraint, i.e., the boundary check [16]. Once PDCE moves these dependent DMA instructions under the boundary check (step ②), ATiM can hoist the invariant branch even higher, outside the loop `j` (step ③). Considering that PDCE is typically excluded from low-level compilers due to its time complexity [29], ATiM's invariant branch hoisting demonstrates the effectiveness of applying aggressive optimizations at a high level tailored to specialized algorithms and hardware. While this optimization does not eliminate or modify the boundary check, it significantly reduces the number

of its dynamic instances (by 40×) and those of DMA and compute operations (by 2×).

5.4 UPMEM Backend and Runtime Support

We extend the TVM C/C++ backend to support UPMEM as target hardware for final code generation from TIR programs to target-specific executables. The backend generates kernel C code with UPMEM built-in functions from optimized TIR programs and handles host code generation by lowering data transfer intrinsics in Section 5.2.2 depending on access patterns. For constant tensors (e.g., weight matrix) reused throughout kernel execution, we allow users to guide the runtime to perform data transfers once before kernel launches, while data transfer intrinsics for vector data are directly compiled to loops with runtime function calls in the host code. Then, we reuse the TIR to LLVM IR lowering pass as it does not require UPMEM-specific handling and can be compiled to any hardware supported by the LLVM compiler [33]. ATiM also provides runtime support for UPMEM host and kernel codes through the TVM runtime. These runtime interfaces include host APIs for UPMEM memory allocation/deallocation, data transfers and synchronizations between the host and DPUs, and resource acquisition for compute units, which are implemented using device driver API provided by the UPMEM SDK.

6 Methodology

We implemented ATiM based on the Apache TVM compiler framework [7] version 0.13.0 (commit 97c5de6). The TVM runtime for UPMEM was developed using "Host and DPU Runtime Library" functions provided by the UPMEM SDK version 2021.3.0. For UPMEM kernel compilation, we used the `dpu-upmem-dpurte-clang` compiler from the SDK, which extends Clang-LLVM compiler version 12 [60]. Kernels were compiled with the `-O2` optimization flag.

UPMEM server, simulator, and benchmark configurations. We used a UPMEM server with dual-socket Intel Xeon Gold 5220R CPUs and 32 ranks of PIM-enabled DIMMs totaling 2048 DPUs (banks) for code development and experiments. For an instruction-level performance breakdown for DPU, which is not available through standard UPMEM SDK profiling tools, we used uPIMulator [24] (commit: 870d916).

We evaluated seven tensor algebra operations and two types of neural network layers, fully-connected (FC) layers and multi-head attention (MHA) layers, that represent the main target operations for PIM for their high memory intensity. Tensor algebra operations are listed below:

- Vector addition (VA): $C(i) = A(i) + B(i)$
- Reduction (RED): $b = \sum_i A(i)$
- Matrix times vector (MTV): $C(i) = A(i, j) \cdot B(j)$
- Tensor times vector (TTV): $C(i, j) = A(i, j, k) \cdot B(k)$
- Multiple matrix times vector (MMTV): $C(i, j) = \sum_k A(i, j, k) \cdot B(i, k)$
- General vector addition (GEVA): $C(i) = c \cdot A(i) + d \cdot B(i)$
- General matrix-vector multiplication (GEMV): $C(i) = c \cdot A(i, j) \cdot B(j)$

We also evaluated the fully connected (MTV) and MMTV operations of the multi-head attention layers of GPT-J 6B and 30B [63], where these operations dominate runtime. The FC layer includes four types of MTV operations: QKV generation, QKV projection, FC, and FC projection. MMTV operations are in the shape of (# batch × # heads, # tokens, 256), with GPT-J 6B using 16 heads and GPT-J 13B using 28 heads. We performed experiments with batch sizes of 1, 4, and 16, and token sizes of 64, 128, 256, and 512 to cover common request sizes used in LLM datasets [53, 58].

Experimental setup. We used the PrIM benchmark [23] as our baseline, designed for UPMEM hardware benchmarking and featuring a wide range of highly hand-optimized kernels. We used VA, RED, and MTV from PrIM [23] and wrote GEVA, TTV, MMTV, and GEMV based on PrIM's codes and programming methodology (PrIM-style). We modified PrIM's VA and MTV to add constant factors for GEVA and GEMV. For TTV, we flattened the outer loop dimension on the host side while using the unmodified MTV kernel for DPU. For MMTV, we adjusted the host code to distribute DPUs across the outer loop dimension.

The following configurations were used to evaluate and compare ATiM's efficiency with prior work in Section 7.1.

- *PrIM/PrIM(E):* PrIM and PrIM-style benchmarks using default parameters from PrIM [23], except for the number of DPUs, which was selected via grid search (2^n where $5 \leq n \leq 11$ for MMTV and $8 \leq n \leq 11$ for the rest).
- *PrIM+search:* PrIM and PrIM-style benchmarks, with the number of DPUs, the number of tasklets, and the WRAM caching tile size determined via grid search. This configuration highlights the importance of parameter searches in general and contrasts independent search spaces with ATiM's joint search space.
- *SimplePIM:* VA and RED kernels provided by SimplePIM [5].
- *ATiM:* Codes compiled by ATiM's search-based code generator as described in Section 5.2.
- *CPU-autotuned:* Tensor programs autotuned for CPU using TVM's MetaSchedule [52] with default configurations.

7 Experimental Results

We evaluated the performance of ATiM-compiled tensor operations on UPMEM and compared it with related work, focusing on the effectiveness of search-based code generation with the joint search space, the impact of each autotuning parameter on workload performance, and the advantages of tensor-level boundary check optimizations. In summary, our experimental results demonstrate:

- Tensor operations auto-tuned with ATiM outperform PrIM, PrIM(E), and SimplePIM by 2.49×, 1.85×, and 2.86× on average, respectively. Even when compared to PrIM+search, ATiM achieves an average speedup of 1.84× by exploring a broader search space.
- ATiM's tensor-level boundary check optimizations effectively handle diverse kernels with varying tensor shapes, delivering performance comparable to or up to 20.5% better than hand-tuned codes (PrIM).
- ATiM outperforms auto-tuned CPU versions for larger tensor sizes up to 23.3×, for which its performance gains increase significantly due to reduced data movement overhead.

7.1 Autotuned Performance of Tensor Programs

VA and GEVA. ATiM improves the performance of VA and GEVA by reducing kernel execution time by 28.2% and 17.8% on average, respectively. While PrIM follows the programming guide's recommendation of using 1024 bytes caching tiles, PrIM+search and ATiM typically select smaller tiles between 64 and 256 bytes identified

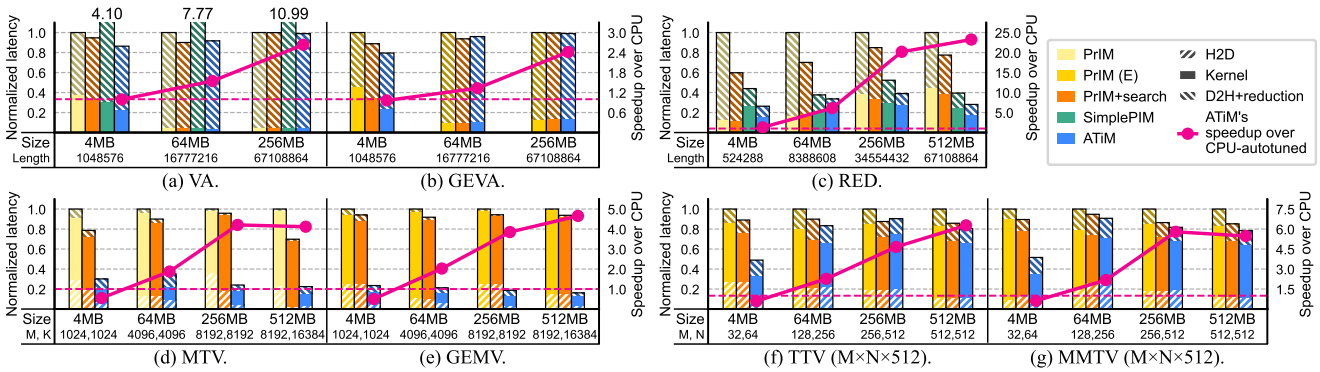


Figure 9: Tensor operation performance (normalized to PrIM).

Table 3: Autotuned parameters for PrIM, PrIM(E), PrIM+search and ATiM. ATiM specifies the number of DPUs per dimension type (spatial, reduction), and PrIM distributes DPUs only on the outermost spatial dimension.

Workload	Size (MB)	PrIM	PrIM+search			ATiM		
		DPUs	DPUs	tasklets	Caching tile size	DPUs /dim	tasklets	Caching tile size
RED	4	256	256	8	32	128	8	32
	64	1024	1024	8	256	512	16	64
	256	1024	2048	16	256	1024	16	128
	512	1024	2048	8	128	2048	16	128
MTV	4	256	256	24	16	(16,16)	16	(32,32,4)
	64	256	256	16	16	(32,64)	16	(64,8,8)
	256	512	256	24	8	(128,16)	16	(8,8,2)
	512	512	512	24	16	(32,64)	16	(64,32,2)
GEMV	4	256	256	8	16	(16,16)	16	(64,32,2)
	64	256	256	8	16	(256,8)	8	(128,64,2)
	256	512	512	8	16	(32,64)	16	(128,32,8)
	512	512	512	8	16	(128,16)	16	(32,16,4)
TTV	4	256	256	8	16	(32,8)	16	(16,16,4)
	64	1024	1024	16	16	(128,8)	16	(64,32,2)
	256	2048	2048	16	16	(2048,1)	16	(64,64,4)
	512	2048	2048	24	8	(512,4)	16	(128,16,8)
MMTV	4	64	64	16	16	(32,8)	16	(128,32,4)
	64	512	512	24	16	(2048,1)	16	(128,64,2)
	256	2048	2048	16	16	(2048,1)	16	(16,16,2)
	512	2048	2048	16	128	(2048,1)	16	(128,128,4)
VA	4	2048	2048	8	16	1024	8	(32,32)
	64	2048	2048	16	32	2048	8	(256,256)
	256	2048	2048	16	64	2048	16	(32,32)
GEVA	4	1024	1024	8	128	1024	16	(64,64)
	64	1024	2048	24	32	2048	8	(32,32)
	256	2048	2048	16	16	2048	16	(16,16)

from searches. Using smaller caching tiles prevents tasklets from becoming idle, thereby enhancing tasklet-level parallelism and more effectively hiding WRAM access latency through pipelined tasklet execution. As shown in Fig. 9(a) and (b), the performance impact is more visible for smaller tensors, as D2H latencies dominate the runtime for larger tensors. ATiM further improves kernel performance by 13.2% compared to PrIM+search by generating intra-DPU DMA instructions with static sizes, thus reducing the number of instructions needed to initiate DMA transfers. In contrast, SimplePIM performs 4-11× worse than PrIM and ATiM due to inefficient D2H transfers, unnecessarily copying the entire tensor on the host side.

RED. For the reduction operation, ATiM significantly outperforms PrIM, PrIM+search, and SimplePIM by 3.19×, 2.31×, and 1.37× on average, respectively, by generating more efficient D2H data transfer and parallel reduction codes. When only one partial reduction result per DPU needs to be transferred to the host, PrIM and PrIM+search redundantly send the results from all tasklets, leading to excessive D2H data movement overhead. While SimplePIM avoids redundant transfers, we found that its DPU partial reduction and host final reduction have inefficient implementations; the host final reduction code incurs extra overhead from invoking multiple internal library functions, while global barriers at each partial reduction step create higher synchronization overhead than the two-thread handshake mechanism used by PrIM(+search) and ATiM.

MTV and GEMV. ATiM achieves speedups up to 6.18× and 5.79× for MTV and GEMV over PrIM and PrIM+search. These performance gains result from a synergetic interplay of multiple autotuning factors, primarily 2D tiling with hierarchical reduction and caching tile location and size optimizations. MTV and GEMV operations have a spatial loop dimension of up to 8192 elements, and if the tiling is performed only on this spatial dimension, there is insufficient inter-DPU parallelism to fully utilize 2048 DPUs. By applying 2D tiling on both spatial and reduction loop dimensions, ATiM generates a sufficient number of smaller tiles to be distributed and computed, significantly reducing the kernel execution time up to 8.84×. As shown in Table 3, ATiM identifies optimal 2D tile sizes (factors between 16 and 256) through autotuning, whereas the prior work relies only on 1D tiling. This also reduces H2D data transfer overheads, with ATiM providing up to 8.37× speedup compared to baseline while slightly increasing post-processing time due to hierarchical reduction. Although it is challenging to isolate individual factors since multiple autotuning parameters interact with each other, the performance gap between PrIM and PrIM+search, both without reduction loop tiling, indicates that caching tile sizes also have a significant performance impact.

TTV and MMTV. TTV and MMTV achieve 2.04× and 1.94× speedups for small tensors, similar to MTV and GEMV, though the performance gains noticeably diminish with large tensors. While these kernels have more spatial loop levels than MTV and GEMV, providing greater inter-DPU and intra-DPU parallelism, the potential for tiling on the reduction loop dimension is limited by smaller reduction loop iterations. As a result, ATiM makes similar DPU distribution decisions as PrIM(+search), exploiting spatial loop tiling only without parallel reduction. ATiM's balanced evolutionary search mechanism ensures schedule candidates without `rfactor` are not dropped early in the autotuning process in this case. On the other hand, for smaller tensors, where the reduction dimension relatively larger than the spatial dimensions, the benefit of reducing data movement through parallel reduction outweighs its overhead. Thus, ATiM leverages additional parallelism on the reduc-

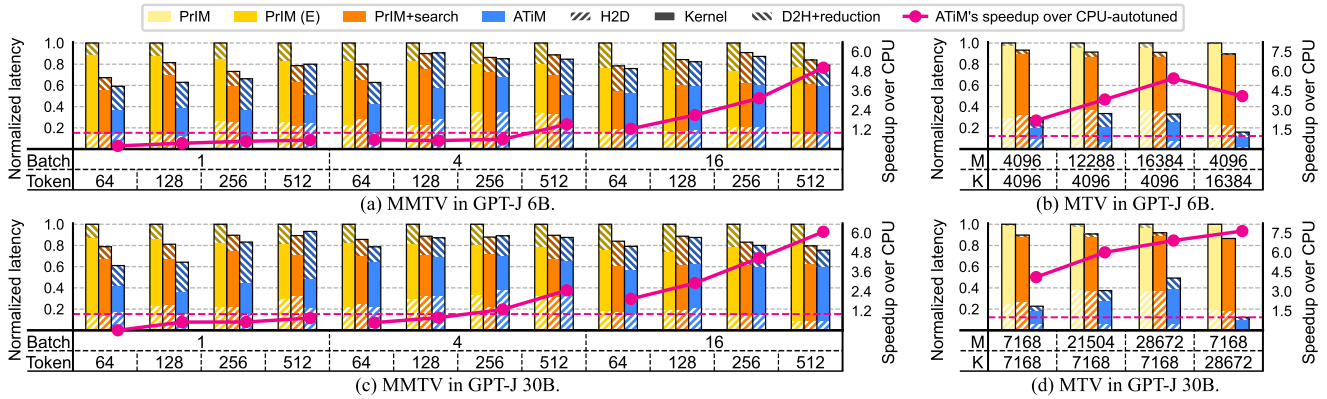


Figure 10: Performance of FC (MTV) and MMTV operations of the MHA layers of GPT-J 6B and 30B (normalized to PrIM).

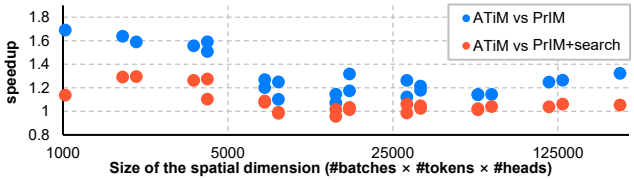


Figure 11: ATiM's speedup for MMTV with varying spatial dimension sizes.

tion loop dimension to further improve H2D and kernel execution time by 1.44× and 3.02×, respectively.

UPMEM vs. CPU. Comparing five PIM versions, including ATiM, with an autotuned CPU version for each tensor program, we found that PIM consistently outperform CPU by up to 23.3× for tensors ≥ 64 MB, while for smaller 4 MB tensors, most PIM versions perform worse than CPUs due to high parallel overheads, except for RED, VA, and GEVA where ATiM performs comparably. For all tensor programs, CPU versions suffer from increased data movement overhead as tensor size grows, while PIM versions benefit from in-place parallel computation across hundreds to thousands of DPUs. For MTV and GEMV, the data movement savings for PrIM(+search) are offset by limited inter-DPU parallelism. Its 1D tiling strategy, which tiles tensor rows only, fails to fully utilize parallel DPUs and generates relatively long kernels with large tiles. In contrast, ATiM significantly outperforms both CPU-autotuned and PrIM(+search) versions by identifying optimal 2D tiling configurations, fully leveraging DPU parallelism and hierarchical reduction.

7.2 Autotuned Performance of GPT-J Layers

We evaluated ATiM's performance on the MTV and MMTV from a real-world model (GPT-J) to assess its practical applicability (Fig. 10).

MMTV in GPT-J. GPT-J has a fixed reduction loop size of 256 elements, but its spatial dimension as a product of the number of heads (depending on the model type), batch size, and number of tokens can vary significantly. Fig. 10 illustrates ATiM's autotuning efficiency across MMTV kernels with different spatial dimension sizes, showing speedups ranging from 7.24% to 69.1%. GPT-6B, with fewer heads, outperforms GPT-30B by 5.72% on average, and both models perform better with a single batch size compared to batch sizes of four and sixteen, achieving average speedups of 20.8% and 11.7%, respectively. As discussed earlier, with smaller spatial dimensions, the inter- and intra-DPU parallelism benefit from reduction dimension tiling becomes more pronounced. Fig. 11 plotting the relationship between spatial dimension sizes and speedups shows that ATiM offers significant advantages over PrIM(+search) for spatial dimensions ≤ 5000, after which speedups plateau. Additionally, ATiM's fine-grained caching size autotuning ability provides further speedups beyond reduction dimension tiling. For example, with GPT-J 6B (batch size of 4 and 64 tokens), ATiM achieves a 59.2% speedup by using two distinct caching tile sizes found through searches, while PrIM+search uses one size.

MTV in GPT-J. ATiM shows significant performance improvements over PrIM and PrIM+search (up to 8.21× and 7.11×) across all four MTV kernels in GPT-J. As discussed in the performance analysis for the MTV kernel, tiling on the reduction loop dimension (column) effectively reduces both kernel execution and data transfer times. Moreover, the benefits of reduction loop tiling increase as the reduction dimension becomes larger relative to the spatial dimension (row), offsetting post-reduction overhead. For instance, ATiM achieves a 3.03× speedup for a 16384×4096 tensor, compared to a 6.25× speedup for its transposed form, 4096×16384.

UPMEM vs. CPU. For the MMTV operations in the MHA layer, ATiM achieves speedups of up to 5.01× and 6.07× over CPU for the 6B and 30B parameter models, respectively. Notably, while ATiM-autotuned versions initially perform slower than CPU-autotuned versions with smaller workloads (up to a batch size of 4 and 256 tokens), they show significant performance improvements beyond this point, scaling proportionally with increasing tensor sizes. We observed that for small tensors, communication reduction overheads outweigh parallelization benefits as the number of DPUs increases. Using fewer DPUs can reduce these overheads but limits its performance due to reduced inter-DPU parallelism. Conversely, with larger tensor dimensions, UPMEM scales more effectively than the CPU by fully exploiting data parallelism across DPUs and better amortizing parallel overheads.

The MTV workload consistently outperforms the CPU across all tensor shapes, achieving speedups of up to 7.65×. Since each tensor size is at least 64MB in this scenario, DRAM bandwidth becomes the primary bottleneck for the CPU. ATiM not only mitigates this bottleneck through in-memory processing but also leverages higher inter-DPU parallelism, delivering greater and more scalable acceleration. Both ATiM and CPU show performance variations for the tensors of the same size with different (transposed) shapes, as shown in the last two columns in Fig. 10(b) and (d).

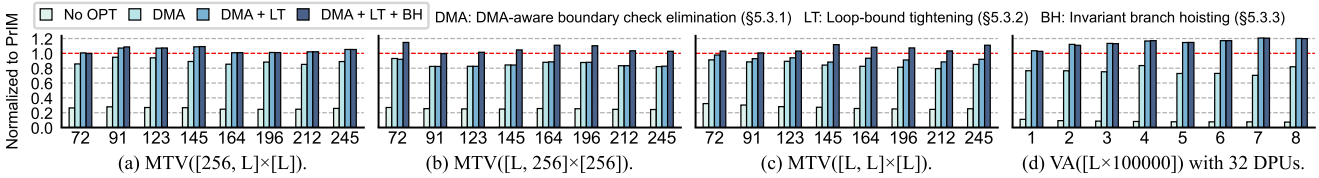


Figure 12: Kernel performance with PIM-aware optimizations in Section 5.3 applied. (a) and (b) are misaligned on one axis, (c) on both axes, and (d) misaligned VA.

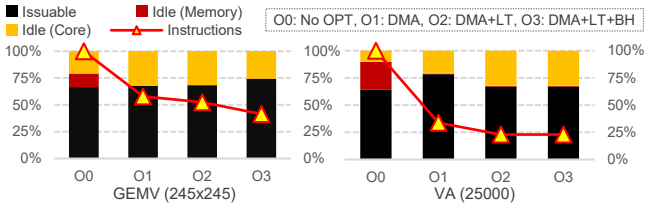


Figure 13: Breakdown of a single DPU's runtime (active in black and idle in red and yellow) under PIM-aware optimizations, as simulated with uPIMulator. The line with triangular markers indicates the number of instructions, normalized to `No-OPT (O0)`.

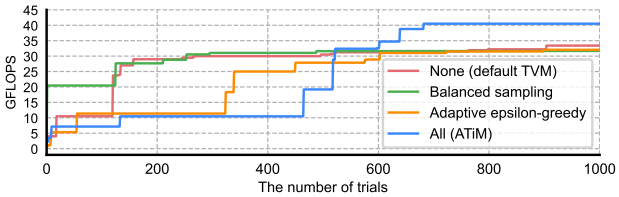


Figure 14: Autotuning efficiency of balanced sampling and adaptive epsilon-greedy strategies (individually and in combination).

7.3 PIM-aware Optimization Performance

Fig. 12 shows the performance of the MTV and VA kernels optimized by different combinations of PIM-aware optimizations in Section 5.3, compared to PrIM versions. When all three optimizations are applied, MTV and VA achieve up to 14.7% and 20.5% (5.37% and 14.4% on average), respectively. For all scenarios, DMA-aware boundary check elimination (`DMA`) provides the largest performance gain by replacing element-wise memory assignment and boundary checks with MRAM-WRAM DMA instructions. Loop-bound tightening (`LT`) further improves performance for workloads with misaligned columns (Fig. 12(a)) by eliminating redundant boundary checks inside computation loops (`k` loop in Fig. 8). Invariant branch hoisting (`BH`) optimizes tensors with misaligned rows (Fig. 12(b)) by lifting invariant branches with row variables (`i` in Fig. 8) outside main loops and skipping unnecessary iterations. For workloads with both row and column misalignments, such as dynamic square shapes in Fig. 12(c), both `LT` and `BH` are applied to reduce branch overheads on all axes. These optimizations are also effective for other kernels; for example, `LT` reduces redundant branches in the VA kernel's computation loop, achieving up to a 20.5% speedup (Fig. 12(d)). In summary, ATiM effectively optimizes loops with boundary checks, generating kernel codes with minimal branch penalties for UPMEM.

We performed additional simulator-based experiments to further analyze how these optimizations impact data movement and kernel execution performance (Fig. 13). The results confirm our assumption that frequent, small DMA requests in `O0` incur significant memory stalls, which are eliminated by DMA-aware boundary-check elimination (`O1`). The subsequent loop-bound tightening (`O2`) and invariant branch hoisting (`O3`) optimizations further decrease the total instruction count by reducing the number of DMA requests, branches, and loop iterations.

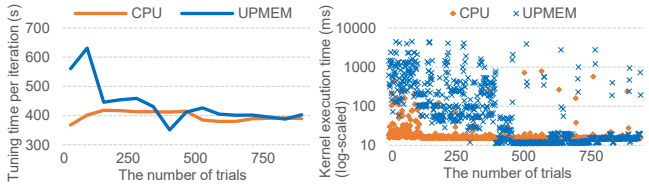


Figure 15: Per-iteration autotuning time (left) and candidate execution time (right) throughout 16 iterations (64 trials each) on CPU and UPMEM (ATiM).

7.4 Balanced Evolutionary Search Result

The combination of balanced sampling and adaptive epsilon-greedy strategy, as detailed in Section 7.4, improves the autotuning process to converge at a higher performance result than TVM's original evolutionary search or standalone application of either technique. While ATiM initially shows lower autotuning efficiency, its balanced exploration of `rfactor` and `non-rfactor` candidates allows it to converge to optimal performance after 400 trials (40% of the total), delivering significant speedups—21.2% over TVM, and 26.4% and 27.9% over each standalone version (Fig. 14). When applied individually, each optimization may penalize early iterations without addressing candidate imbalances, ultimately performing similarly to the default TVM.

8 Discussion

Autotuning overheads. Compared to the CPU autotuning time with the original TVM compiler, the UPMEM autotuning time with ATiM is 11.5% and 18.5% higher for the MTV and MMTV kernels with 4, 64, 256, and 512 MB tensors for 1000 trials. A detailed analysis of the autotuning time throughout iterations reveals that ATiM spends significantly more time on hardware measurements than the CPU while it eventually converges to a better-performing version. ATiM's balanced evolutionary search prioritizes exploration in the early autotuning iterations to cover a larger search space than CPUs, which results in higher initial autotuning overheads (left graph of Fig. 15). Additionally, we observed that UPMEM shows greater performance variability with suboptimal DPU tiling configurations, while CPU performance remains relatively stable, as shown in the outlier candidates (blue dots in the right graph of Fig. 15). We plan to address these issues by refining the search algorithm and cost model in future work.

Extension to other DRAM-PIM architectures. ATiM is designed to support DRAM-PIM architectures with bank-level compute units by exploiting tensor-level IR for common abstractions and specializing the TIR lowering passes for specific hardware targets. For DRAM-PIM with MAC acceleration logic instead of general-purpose processors [35, 36], ATiM's autotuner and code generator would need to be extended to handle different bank mapping policies and

compile to PIM-specific vector intrinsics or tensorized functions. For instance, HBM-PIM assigns a processing unit (PU) to every two banks and activates it with memory commands in a special mode to perform matrix-vector operations with data from memory rows and PU registers [35]. To support this architecture, ATiM must redefine the DPU binding in two levels – one at the bank level and another at the PU level – and support specialized vector intrinsic using TVM's tensorization feature [7] to enforce compatible loop structures. Our preliminary results using an open-source HBM-PIM simulator [51] and library [50] suggest that ATiM can be seamlessly extended to support an automated host and kernel code generation for HBM-PIM. Future work includes evolving ATiM into an extensible and portable tensor compiler for diverse DRAM-PIM hardware.

DL framework interfaces. To interface with DL frameworks and automate end-to-end compilation from input models, it is necessary to extend the graph-level TVM frontend using Relax [32] or Relay [48] IR to invoke ATiM and update the TVM runtime to manage layer and memory mapping to DRAM-PIM hardware. Once trained models are imported from other DL frameworks, such as TensorFlow [1], PyTorch [45], and ONNX [3], by the TVM frontend, ATiM can further perform inter-kernel optimizations at the graph IR level, e.g., layer fusion and reordering, to reduce inter-DPU data transfers and DPU kernel launch overheads. Integrating ATiM with application interfaces and optimizing its system-level usage is an important direction for our future work.

Integration with MLIR. MLIR [34] can serve as an alternative high-level IR for PIM compilers [26]. While ATiM relies on TVM's autotuning framework, its core techniques, such as schedule primitives for data distribution and kernel optimizations, can be incorporated into MLIR's transform dialect [40], which enables dialect-to-dialect transformations. For example, GPU programs in MLIR GPU dialects (e.g., Triton [59]) could be lowered and optimized for PIM execution by defining transformation rules. Fully integrating ATiM with MLIR would require independent research, involving the development of cost-based search algorithms and significant dialect extensions.

Broader application of DMA and boundary optimizations. The PIM-aware optimizations in Section 5.3 can be generalized beyond UPMEM, benefiting hardware with similar architectural characteristics. Specifically, DMA-aware boundary check elimination, which enables safe data overfetching via DMA, aligns well with accelerators that support efficient bulk memory transfers [10, 62, 64]. Loop-bound tightening and invariant branch hoisting (Sections 5.3.2 and 5.3.3) can effectively improve performance for resource-constrained processors, such as RISC-V and ARM-based edge CPUs, which are widely used in embedded AI and IoT devices yet lack advanced branch prediction and other latency-hiding mechanisms [2, 56, 65]. As future work, we plan to develop abstractions and algorithms to enable more portable application of these optimizations across a broader range of hardware targets.

9 Related Work

Tensor program support for DRAM-PIM. Recent research has actively explored software support for DRAM-PIM hardware [5, 21, 25, 26, 31, 38, 44, 49, 55]. Vendor-provided solutions [31, 49] offer full software stacks but rely heavily on hand-optimized libraries, highlighting the need for optimizing code generators like ATiM for tensor programs. Several efforts proposed DRAM-PIM specific compilation and optimization support but have limitations compared to ATiM. For instance, CINM [26] defines MLIR dialects for various PIM architectures but lacks systematic optimization and auto-tuning capabilities. iPIM [21] employs a scheduling language for efficient data mapping but supports limited data distribution strategies tailored to the cube-structured memory of HMC [46]. SimplePIM [5] provides primitives for UPMEM but is restricted to 1D arrays, limiting its support for general tensor operations. PIMFlow [55] extends TVM for CNN models on DRAM-PIM but is not fully integrated with TVM's runtime and autotuner, relying on fixed optimization strategies. Other studies, TC-CIM [15], target specific PIM hardware like Computing-in-Memory (CIM), focusing on tensor mapping that considers its analog computation nature. In contrast, ATiM jointly optimizes host and kernel operations for DRAM-PIM architectures, offering a more flexible and comprehensive approach.

PIM-specific code optimizations. Prior work has shown that host data distribution and kernel multi-level tiling are crucial for achieving high performance in DRAM-PIM systems [17, 25, 66]. Studies such as [17, 26] revealed that efficient data distribution can reduce data movement and enhance DPU parallelism, improving UPMEM performance. While these works explored a limited set of kernel optimization schemes, ATiM systematically explores the host and kernel loop optimization space. PIMnast [25] examined performance-tuning factors for DRAM-PIM hardware, highlighting how memory configuration and operation requirements influence data distribution strategies and kernel optimizations through tiling and caching strategies. Building upon these insights, ATiM implements a search-based code generation mechanism with PIM-specific autotuning parameters. Additionally, DietCode [66] proposed algorithm-level optimizations to reduce boundary check overheads, some of which have been incorporated into TVM. ATiM leverages these techniques to introduce more aggressive optimizations tailored to UPMEM hardware constraints and tensor loop structures.

10 Conclusion and Future Work

ATiM tackles the challenge of building a fully automated code generation support for emerging DRAM-centric accelerators while efficiently navigating the vast space of host and kernel optimizations. It carefully expands existing tensor compiler infrastructure to provide flexible, autotuning-driven code generation for UPMEM, complying with its tensor language semantics and seamlessly integrating with its lowering pipeline. Experimental results show ATiM's strong potential as a practical, optimizing tensor compiler for diverse DRAM-PIM architectures. For our future work, we will focus on developing ATiM's application-level interfaces, supporting other types of DRAM-PIM hardware, and enabling graph-level autotuning.

Acknowledgments

We thank the anonymous reviewers for their valuable feedback. This work was supported in part by Samsung Advanced Institute of Technology (SAIT) and in part by Institute for Information & communications Technology Planning & Evaluation (IITP) grants (RS-2019-II191906, 2021-0-00871, 2021-0-00310, RS-2023-00228970) and National Research Foundation (NRF) grants (RS-2023-00222663, RS-2023-00283799) funded by the Korean government (MSIT).

References

[1] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, et al. 2016. {TensorFlow}: a system for {Large-Scale} machine learning. In *12th USENIX symposium on operating systems design and implementation (OSDI 16)*. 265–283.

[2] Krste Asanovic, Rimas Avizienis, Jonathan Bachrach, Scott Beamer, David Biancolin, Christopher Celio, Henry Cook, Daniel Dabbelt, John Hauser, Adam Izraelevitz, et al. 2016. The rocket chip generator. *EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2016-17* 4 (2016), 6–2.

[3] Junjie Bai, Fang Lu, Ke Zhang, et al. 2019. ONNX: Open Neural Network Exchange. https://github.com/onnx/onnx.

[4] Arthur Bernhardt, Andreas Koch, and Ilia Petrov. 2023. pimDB: From Main-Memory DBMS to Processing-In-Memory DBMS-Engines on Intelligent Memories. In *Proceedings of the 19th International Workshop on Data Management on New Hardware* (Seattle, WA, USA) *(DaMoN '23)*. Association for Computing Machinery, New York, NY, USA, 44–52. https://doi.org/10.1145/3592980.3595312

[5] J. Chen, J. Gomez-Luna, I. El Hajj, Y. Guo, and O. Mutlu. 2023. SimplePIM: A Software Framework for Productive and Efficient Processing-in-Memory. In *2023 32nd International Conference on Parallel Architectures and Compilation Techniques (PACT)*. IEEE Computer Society, Los Alamitos, CA, USA, 99–111. https://doi.org/10.1109/PACT58117.2023.00017

[6] Liang-Chi Chen, Chien-Chung Ho, and Yuan-Hao Chang. 2023. UpPipe: A Novel Pipeline Management on In-Memory Processors for RNA-seq Quantification. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*. 1–6. https://doi.org/10.1109/DAC56929.2023.10247915

[7] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. USENIX Association, Carlsbad, CA, 578–594. https://www.usenix.org/conference/osdi18/presentation/chen

[8] Tianqi Chen, Lianmin Zheng, Eddie Yan, Ziheng Jiang, Thierry Moreau, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. Learning to Optimize Tensor Programs. In *Advances in Neural Information Processing Systems*, S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett (Eds.), Vol. 31. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2018/file/8b5700012be65c9da25f49408d959ca0-Paper.pdf

[9] Benjamin Y. Cho, Jeageun Jung, and Mattan Erez. 2021. Accelerating bandwidth-bound deep learning inference with main-memory accelerators. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis* (St. Louis, Missouri) *(SC '21)*. Association for Computing Machinery, New York, NY, USA, Article 44, 14 pages. https://doi.org/10.1145/3458817.3476146

[10] Lucian Codrescu, Willie Anderson, Suresh Venkumanhanti, Mao Zeng, Erich Plondke, Chris Koob, Ajay Ingle, Charles Tabony, and Rick Maule. 2014. Hexagon DSP: An architecture optimized for mobile multimedia and communications. *IEEE Micro* 34, 2 (2014), 34–43.

[11] Prangon Das, Purab Ranjan Sutradhar, Mark Indovina, Sai Manoj Pudukotai Dinakarrao, and Amlan Ganguly. 2022. Implementation and Evaluation of Deep Neural Networks in Commercially Available Processing in Memory Hardware. In *2022 IEEE 35th International System-on-Chip Conference (SOCC)*. 1–6. https://doi.org/10.1109/SOCC56010.2022.9908126

[12] Fabrice Devaux. 2019. The true Processing In Memory accelerator. In *2019 IEEE Hot Chips 31 Symposium (HCS)*. 1–24. https://doi.org/10.1109/HOTCHIPS.2019.8875680

[13] Fabrice Devaux. 2019. The true Processing In Memory accelerator. In *2019 IEEE Hot Chips 31 Symposium (HCS)*. 1–24. https://doi.org/10.1109/HOTCHIPS.2019.8875680

[14] Jeff Draper, Jacqueline Chame, Mary Hall, Craig Steele, Tim Barrett, Jeff LaCoss, John Granacki, Jaewook Shin, Chun Chen, Chang Woo Kang, Ihn Kim, and Gokhan Daglikoca. 2002. The architecture of the DIVA processing-in-memory chip. In *Proceedings of the 16th International Conference on Supercomputing* (New York, New York, USA) *(ICS '02)*. Association for Computing Machinery, New York, NY, USA, 14–25. https://doi.org/10.1145/514191.514197

[15] Andi Drebes, Lorenzo Chelini, Oleksandr Zinenko, Albert Cohen, Henk Corporaal, Tobias Grosser, Kanishkan Vadivel, and Nicolas Vasilache. 2020. TC-CIM: Empowering Tensor Comprehensions for Computing-In-Memory. http://impact.gforge.inria.fr/impact2020/ 10th International Workshop on Polyhedral Compilation Techniques, IMPACT 2010 ; Conference date: 22-01-2020 Through 22-01-2020.

[16] Siyuan Feng, Bohan Hou, Hongyi Jin, Wuwei Lin, Junru Shao, Ruihang Lai, Zihao Ye, Lianmin Zheng, Cody Hao Yu, Yong Yu, and Tianqi Chen. 2023. TensorIR: An Abstraction for Automatic Tensorized Program Optimization. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (Vancouver, BC, Canada) *(ASPLOS 2023)*. Association for Computing Machinery, New York, NY, USA, 804–817. https://doi.org/10.1145/3575693.3576933

[17] Christina Giannoula, Ivan Fernandez, Juan Gómez-Luna, Nectarios Koziris, Georgios Goumas, and Onur Mutlu. 2022. SparseP: Efficient Sparse Matrix Vector Multiplication on Real Processing-In-Memory Architectures. In *2022 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*. 288–291. https://doi.org/10.1109/ISVLSI54635.2022.00063

[18] Kailash Gogineni, Sai Santosh Dayapule, Juan Gómez-Luna, Karthikeya Gogineni, Peng Wei, Tian Lan, Mohammad Sadrosadati, Onur Mutlu, and Guru Venkataramani. 2024. SwiftRL: Towards Efficient Reinforcement Learning on Real Processing-In-Memory Systems. In *2024 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. 217–229. https://doi.org/10.1109/ISPASS61541.2024.00029

[19] M. Gokhale, B. Holmes, and K. Iobst. 1995. Processing in memory: the Terasys massively parallel PIM array. *Computer* 28, 4 (1995), 23–31. https://doi.org/10.1109/2.375174

[20] Tobias Grosser, Armin Groesslinger, and Christian Lengauer. 2012. Polly—performing polyhedral optimizations on a low-level intermediate representation. *Parallel Processing Letters* 22, 04 (2012), 1250010.

[21] Peng Gu, Xinfeng Xie, Yufei Ding, Guoyang Chen, Weifeng Zhang, Dimin Niu, and Yuan Xie. 2020. iPIM: Programmable In-Memory Image Processing Accelerator Using Near-Bank Architecture. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. 804–817. https://doi.org/10.1109/ISCA45697.2020.00071

[22] Juan Gómez-Luna, Yuxin Guo, Sylvan Brocard, Julien Legriel, Remy Cimadomo, Geraldo F. Oliveira, Gagandeep Singh, and Onur Mutlu. 2023. Evaluating Machine LearningWorkloads on Memory-Centric Computing Systems. In *2023 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. 35–49. https://doi.org/10.1109/ISPASS57527.2023.00013

[23] Juan Gómez-Luna, Izzat El Hajj, Ivan Fernandez, Christina Giannoula, Geraldo F. Oliveira, and Onur Mutlu. 2022. Benchmarking a New Paradigm: Experimental Analysis and Characterization of a Real Processing-in-Memory System. *IEEE Access* 10 (2022), 52565–52608. https://doi.org/10.1109/ACCESS.2022.3174101

[24] Bongjoon Hyun, Taehun Kim, Dongjae Lee, and Minsoo Rhu. 2024. Pathfinding Future PIM Architectures by Demystifying a Commercial PIM Technology. In *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 263–279.

[25] Mohamed Assem Ibrahim, Mahzabeen Islam, and Shaizeen Aga. 2024. Balanced Data Placement for GEMV Acceleration with Processing-In-Memory. arXiv:2403.20297 [cs.AR]

[26] Asif Ali Khan, Hamid Farzaneh, Karl F. A. Friebel, Clément Fournier, Lorenzo Chelini, and Jeronimo Castrillon. 2023. CINM (Cinnamon): A Compilation Infrastructure for Heterogeneous Compute In-Memory and Compute Near-Memory Paradigms. arXiv:2301.07486 [cs.AR]

[27] Jin Hyun Kim, Shin-Haeng Kang, Sukhan Lee, Hyeonsu Kim, Yuhwan Ro, Seungwon Lee, David Wang, Jihyun Choi, Jinin So, YeonGon Cho, JoonHo Song, Jeonghyeon Cho, Kyomin Sohn, and Nam Sung Kim. 2022. Aquabolt-XL HBM2-PIM, LPDDR5-PIM With In-Memory Processing, and AXDIMM With Acceleration Buffer. *IEEE Micro* 42, 3 (2022), 20–30. https://doi.org/10.1109/MM.2022.3164651

[28] Jin Hyun Kim, Shin-haeng Kang, Sukhan Lee, Hyeonsu Kim, Woongjae Song, Yuhwan Ro, Seungwon Lee, David Wang, Hyunsung Shin, Bengseng Phuah, Jihyun Choi, Jinin So, YeonGon Cho, JoonHo Song, Jangseok Choi, Jeonghyeon Cho, Kyomin Sohn, Youngsoo Sohn, Kwangil Park, and Nam Sung Kim. 2021. Aquabolt-XL: Samsung HBM2-PIM with in-memory processing for ML accelerators and beyond. In *2021 IEEE Hot Chips 33 Symposium (HCS)*. 1–26. https://doi.org/10.1109/HCS52781.2021.9567191

[29] Jens Knoop, Oliver Rüthing, and Bernhard Steffen. 1994. Partial dead code elimination. *ACM Sigplan Notices* 29, 6 (1994), 147–158.

[30] Michael Kruse and Hal Finkel. 2018. User-Directed Loop-Transformations in Clang. In *2018 IEEE/ACM 5th Workshop on the LLVM Compiler Infrastructure in HPC (LLVM-HPC)*. 49–58. https://doi.org/10.1109/LLVM-HPC.2018.8639402

[31] Yongkee Kwon, Kornijcuk Vladimir, Nahsung Kim, Woojae Shin, Jongsoon Won, Minkyu Lee, Hyunha Joo, Haerang Choi, Guhyun Kim, Byeongju An, Jeongbin Kim, Jaewook Lee, Ilkon Kim, Jaehan Park, Chanwook Park, Yosub Song, Byeongsu Yang, Hyungdeok Lee, Seho Kim, Daehan Kwon, Seongju Lee, Kyuyoung Kim, Sanghoon Oh, Joonhong Park, Gimoon Hong, Dongyoon Ka, Kyudong Hwang, Jeongje Park, Kyeongpil Kang, Jungyeon Kim, Junyeol Jeon, Myeongjun Lee, Minyoung Shin, Minhwan Shin, Jaekyung Cha, Changson Jung, Kijoon Chang, Chunseok Jeong, Euicheol Lim, Il Park, Junhyun Chun, and Sk Hynix. 2022. System Architecture and Software Stack for GDDR6-AiM. In *2022 IEEE Hot Chips 34 Symposium (HCS)*. 1–25. https://doi.org/10.1109/HCS55958.2022.9895629

[32] Ruihang Lai, Junru Shao, Siyuan Feng, Steven S Lyubomirsky, Bohan Hou, Wuwei Lin, Zihao Ye, Hongyi Jin, Yuchen Jin, Jiawei Liu, et al. 2023. Relax: Composable Abstractions for End-to-End Dynamic Machine Learning. *arXiv preprint arXiv:2311.02103* (2023).

[33] C. Lattner and V. Adve. 2004. LLVM: a compilation framework for lifelong program analysis & transformation. In *International Symposium on Code Generation and Optimization, 2004. CGO 2004.* 75–86. https://doi.org/10.1109/CGO.2004.1281665

[34] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. 2021. MLIR: Scaling Compiler Infrastructure for Domain Specific Com-

putation. In *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. 2–14. https://doi.org/10.1109/CGO51591.2021.9370308
[35] Sukhan Lee, Shin-haeng Kang, Jaehoon Lee, Hyeonsu Kim, Eojin Lee, Seungwoo Seo, Hosang Yoon, Seungwon Lee, Kyounghwan Lim, Hyunsung Shin, Jinhyun Kim, O Seongil, Anand Iyer, David Wang, Kyomin Sohn, and Nam Sung Kim. 2021. Hardware Architecture and Software Stack for PIM Based on Commercial DRAM Technology : Industrial Product. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. 43–56. https://doi.org/10.1109/ISCA52012.2021.00013
[36] Seongju Lee, Kyuyoung Kim, Sanghoon Oh, Joonhong Park, Gimoon Hong, Dongyoon Ka, Kyudong Hwang, Jeongje Park, Kyeongpil Kang, Jungyeon Kim, Junyeol Jeon, Nahsung Kim, Yongkee Kwon, Kornijcuk Vladimir, Woojae Shin, Jongsoon Won, Minkyu Lee, Hyunha Joo, Haerang Choi, Jaewook Lee, Donguc Ko, Younggun Jun, Keewon Cho, Ilwoong Kim, Choungki Song, Chunseok Jeong, Daehan Kwon, Jieun Jang, Il Park, Junhyun Chun, and Joohwan Cho. 2022. A 1ynm 1.25V 8Gb, 16Gb/s/pin GDDR6-based Accelerator-in-Memory supporting 1TFLOPS MAC Operation and Various Activation Functions for Deep-Learning Applications. In *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, Vol. 65. 1–3. https://doi.org/10.1109/ISSCC42614.2022.9731711
[37] Chaemin Lim, Suhyun Lee, Jinwoo Choi, Jounghoo Lee, Seongyeon Park, Hanjun Kim, Jinho Lee, and Youngsok Kim. 2023. Design and Analysis of a Processing-in-DIMM Join Algorithm: A Case Study with UPMEM DIMMs. *Proc. ACM Manag. Data* 1, 2, Article 113 (June 2023), 27 pages. https://doi.org/10.1145/3589258
[38] Junfeng Lin, Huanyu Qu, Songchen Ma, Xinglong Ji, Hongyi Li, Xiaochuan Li, Chenhang Song, and Weihao Zhang. 2023. SongC: A Compiler for Hybrid Near-Memory and In-Memory Many-Core Architecture. *IEEE Trans. Comput.* (2023), 1–14. https://doi.org/10.1109/TC.2023.3311948
[39] LLVM. [n. d.]. LLVM's Analysis and Transform Passes. https://llvm.org/docs/Passes.html.
[40] Martin Paul Lücke, Oleksandr Zinenko, William S Moses, Michel Steuwer, and Albert Cohen. 2024. The MLIR Transform Dialect. Your compiler is more powerful than you think. *arXiv preprint arXiv:2409.03864* (2024).
[41] Onur Mutlu, Saugata Ghose, Juan Gómez-Luna, and Rachata Ausavarungnirun. 2022. A modern primer on processing in memory. In *Emerging Computing: From Devices to Systems: Looking Beyond Moore and Von Neumann*. Springer, 171–243.
[42] R. Nair, S. F. Antao, C. Bertolli, P. Bose, J. R. Brunheroto, T. Chen, C.-Y. Cher, C. H. A. Costa, J. Doi, C. Evangelinos, B. M. Fleischer, T. W. Fox, D. S. Gallo, L. Grinberg, J. A. Gunnels, A. C. Jacob, P. Jacob, H. M. Jacobson, T. Karkhanis, C. Kim, J. H. Moreno, J. K. O'Brien, M. Ohmacht, Y. Park, D. A. Prener, B. S. Rosenburg, K. D. Ryu, O. Sallenave, M. J. Serrano, P. D. M. Siegl, K. Sugavanam, and Z. Sura. 2015. Active Memory Cube: A processing-in-memory architecture for exascale systems. *IBM Journal of Research and Development* 59, 2/3 (2015), 17:1–17:14. https://doi.org/10.1147/JRD.2015.2409732
[43] John Nickolls, Ian Buck, Michael Garland, and Kevin Skadron. 2008. Scalable parallel programming with CUDA. In *ACM SIGGRAPH 2008 Classes* (Los Angeles, California) *(SIGGRAPH '08)*. Association for Computing Machinery, New York, NY, USA, Article 16, 14 pages. https://doi.org/10.1145/1401132.1401152
[44] G. F. Oliveira, A. Olgun, A. Yaglikci, F. Bostanci, J. Gomez-Luna, S. Ghose, and O. Mutlu. 2024. MIMDRAM: An End-to-End Processing-Using-DRAM System for High-Throughput, Energy-Efficient and Programmer-Transparent Multiple-Instruction Multiple-Data Computing. In *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE Computer Society, Los Alamitos, CA, USA, 186–203. https://doi.org/10.1109/HPCA57654.2024.00024
[45] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. *PyTorch: an imperative style, high-performance deep learning library*. Curran Associates Inc., Red Hook, NY, USA.
[46] J. Thomas Pawlowski. 2011. Hybrid memory cube (HMC). In *2011 IEEE Hot Chips 23 Symposium (HCS)*. 1–24. https://doi.org/10.1109/HOTCHIPS.2011.7477494
[47] Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. 2013. Halide: a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Seattle, Washington, USA) *(PLDI '13)*. Association for Computing Machinery, New York, NY, USA, 519–530. https://doi.org/10.1145/2491956.2462176
[48] Jared Roesch, Steven Lyubomirsky, Logan Weber, Josh Pollock, Marisa Kirisame, Tianqi Chen, and Zachary Tatlock. 2018. Relay: a new IR for machine learning frameworks. In *Proceedings of the 2nd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages* (Philadelphia, PA, USA) *(MAPL 2018)*. Association for Computing Machinery, New York, NY, USA, 58–68. https://doi.org/10.1145/3211346.3211348
[49] Samsung Advanced Institute of Technology. 2022. *OneMCC*. https://github.com/SAITPublic/OneMCC
[50] Samsung Advanced Institute of Technology. 2022. *PIMLibrary*. https://github.com/SAITPublic/PIMLibrary
[51] Samsung Advanced Institute of Technology. 2022. *PIMSimulator*. https://github.com/SAITPublic/PIMSimulator
[52] Junru Shao, Xiyou Zhou, Siyuan Feng, Bohan Hou, Ruihang Lai, Hongyi Jin, Wuwei Lin, Masahiro Masuda, Cody Hao Yu, and Tianqi Chen. 2022. Tensor Program Optimization with Probabilistic Programs. In *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh (Eds.), Vol. 35. Curran Associates, Inc., 35783–35796. https://proceedings.neurips.cc/paper_files/paper/2022/file/e894eafae43e68b4c8dfdacf742bcbf3-Paper-Conference.pdf
[53] ShareGPT Team. 2023. *ShareGPT*. https://sharegpt.com,2023
[54] Haichen Shen, Jared Roesch, Zhi Chen, Wei Chen, Yong Wu, Mu Li, Vin Sharma, Zachary Tatlock, and Yida Wang. 2021. Nimble: Efficiently Compiling Dynamic Neural Networks for Model Inference. In *Proceedings of Machine Learning and Systems*, A. Smola, A. Dimakis, and I. Stoica (Eds.), Vol. 3. 208–222. https://proceedings.mlsys.org/paper_files/paper/2021/file/5b47430e24a5a1f9fe21f0e8eb814131-Paper.pdf
[55] Yongwon Shin, Juseong Park, Sungjun Cho, and Hyojin Sung. 2023. PIMFlow: Compiler and Runtime Support for CNN Models on Processing-in-Memory DRAM. In *Proceedings of the 21st ACM/IEEE International Symposium on Code Generation and Optimization* (<conf-loc>, <city>Montréal</city>, <state>QC</state>, <country>Canada</country>, </conf-loc>) *(CGO 2023)*. Association for Computing Machinery, New York, NY, USA, 249–262. https://doi.org/10.1145/3579990.3580009
[56] SiFive. 2021. SiFive E21 Core Complex Manual.
[57] John E. Stone, David Gohara, and Guochun Shi. 2010. OpenCL: A Parallel Programming Standard for Heterogeneous Computing Systems. *Computing in Science & Engineering* 12, 3 (2010), 66–73. https://doi.org/10.1109/MCSE.2010.69
[58] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. 2023. *Stanford Alpaca: An Instruction-following LLaMA model*.
[59] Philippe Tillet, H. T. Kung, and David Cox. 2019. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages* (Phoenix, AZ, USA) *(MAPL 2019)*. Association for Computing Machinery, New York, NY, USA, 10–19. https://doi.org/10.1145/3315508.3329973
[60] UPMEM. [n. d.]. llvm-project. https://github.com/upmem/llvm-project.
[61] Nicolas Vasilache, Oleksandr Zinenko, Theodoros Theodoridis, Priya Goyal, Zachary DeVito, William S. Moses, Sven Verdoolaege, Andrew Adams, and Albert Cohen. 2018. Tensor Comprehensions: Framework-Agnostic High-Performance Machine Learning Abstractions. arXiv:1802.04730 [cs.PL]
[62] Pirmin Vogel, Andrea Marongiu, and Luca Benini. 2018. Exploring shared virtual memory for FPGA accelerators with a configurable IOMMU. *IEEE Trans. Comput.* 68, 4 (2018), 510–525.
[63] Ben Wang. 2021. *Mesh-Transformer-JAX: Model-Parallel Implementation of Transformer Language Model with JAX*.
[64] XILINX. 2020. Versal ACAP AI Engine Architecture Manual.
[65] Joseph Yiu. 2009. *The definitive guide to the ARM Cortex-M3*. Newnes.
[66] Bojian Zheng, Ziheng Jiang, Cody Hao Yu, Haichen Shen, Josh Fromm, Yizhi Liu, Yida Wang, Luis Ceze, Tianqi Chen, and Gennady Pekhimenko. 2022. DietCode: Automatic optimization for dynamic tensor program. (2022).
[67] Lianmin Zheng, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, Jun Yang, Danyang Zhuo, Koushik Sen, Joseph E. Gonzalez, and Ion Stoica. 2020. Ansor: Generating High-Performance Tensor Programs for Deep Learning. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, 863–879. https://www.usenix.org/conference/osdi20/presentation/zheng

A Artifact Appendix

A.1 Abstract

Our artifact includes setup and evaluation environments for the five configurations in Section 6: *CPU-autotuned*, *PrIM/(E)*, *PrIM+search*, *SimplePIM*, and *ATiM*, along with the full implementation of ATiM, including source code. Scripts for configuration setup, autotuning, and evaluation are provided for each configuration. These scripts can be used to reproduce the main results presented in the paper (Figs. 9, 10, and 12). To enable quick reproduction of results, we provide pre-autotuned ATiM modules for tensor programs. Users can also generate new ATiM modules by following the instructions provided below. The top-level scripts are implemented by composing low-level scripts for an extensible and customizable evaluation workflow (see Section A.5). Reproducing the experiments in Section 7 requires access to a UPMEM server.

A.2 Artifact check-list (meta-information)

- **Algorithm:** Includes methods for (1) autotuning using ATiM-extended sketch generation rules and evolutionary search, (2) generating host and DPU code, including data transfer code, and (3) performing PIM-aware optimizations.
- **Compilation:** Uses the `dpu-upmem-dpurte-clang` compiler from UPMEM SDK version 2021.3.0 with `-O2` optimization, LLVM 16.0.6 for TVM compilation and CPU code generation, and a modified Apache TVM compiler (based on version 0.13.0, commit 97c5de6) for ATiM.
- **Model:** MTV and MMTV layers in the GPT-J.
- **Run-time environment:** Linux (Ubuntu 20.04) with UPMEM driver, backends (communication library), and DPU runtime included in the UPMEM SDK version 2021.3.0.
- **Hardware:** Intel Xeon Gold 5220R CPU server equipped with 20 DDR4-2400 PIM modules. The server requires UPMEM firmware (BIOS and MCU) provided by the UPMEM SDK.
- **Metrics:** Execution time.
- **Output:** Optimal tunable parameters for CPU-autotuned, PrIM/(E), PrIM+search, and SimplePIM and ATiM modules. Experimental results are stored as CSV files, and our scripts generate corresponding graphs in PDF format.
- **Experiments:** Detailed experimental procedures are described in Section A.5 and the provided `README.md` file.
- **How much disk space required (approximately)?:** 10 GB.
- **How much time is needed to prepare workflow (approximately)?:** 2 days to prepare tuned binaries and modules for PrIM/(E), PrIM+search, SimplePIM, CPU autotuning, and ATiM.
- **How much time is needed to complete experiments (approximately)?:** 30 minutes.
- **Publicly available?:** Yes (https://github.com/SNU-CODElab/atim).
- **Workflow automation framework used?:** No.
- **Archived (provide DOI)?:** https://doi.org/10.5281/zenodo.15379924.

A.3 Description

A.3.1 How to access. Our source code and experimental scripts are available on Zenodo: https://doi.org/10.5281/zenodo.15379924. A GitHub repository is also provided (https://github.com/SNU-CODElab/atim).

A.3.2 Hardware dependencies. We recommend using UPMEM systems with at least 16 PIM modules (2048 DPUs) to ensure sufficient inter-DPU parallelism. Our experiments use DDR4-2400 PIM modules, which are currently the only available UPMEM PIM modules.

A.3.3 Software dependencies. We implemented and tested our codes on the Ubuntu 20.04 x86-64 system with UPMEM SDK version 2021.3.0. Additional software dependencies include LLVM and TVM prerequisites on Ubuntu. Specific software versions include Python 3.11, CMake 3.24 or higher, NumPy 1.26.4, XGBoost 1.6.1, and LLVM 16 (including clang). We strongly recommend using the Docker image `yongwonshin/atim:v0.1` or later, as it includes all software dependencies and prerequisites required for running experiments.

A.4 Installation

A.4.1 Using a Docker image. The *docker* package is required. Activate the Docker image and install ATiM with the following commands:

```
docker run -it --privileged yongwonshin/atim:v0.1
./install.sh
```

A.4.2 Local Installation. ATiM source codes including PrIM and SimplePIM can be obtained via Zenodo or GitHub.

```
# Extract from Zenodo:
wget -O atim.zip https://zenodo.org/records/15379924/
     files/atim.zip?download=1
unzip atim.zip

# Or clone from GitHub:
git clone -b artifact https://github.com/SNU-CODElab/atim
     .git
```

Detailed installation instructions are available in the `README.md` file in each archive.

A.5 Experiment workflow

The experiment workflow consists of executing parameter searches or autotuning processes for optimal tunable parameters and using them to compile tensor programs for each configuration. Users may skip this step and directly proceed to Section A.6 by reusing pre-tuned parameters and modules included in the artifact archive. Users can follow the steps below to perform autotuning for CPU baseline and ATiM and parameter searches for PrIM/(E), PrIM+search, and SimplePIM:

```
# Step 0: prepare for autotuning
cd atim/evaluation
conda activate atim-venv

# Step 1: perform autotuning for CPU-autotune
python cpu_autotune.py

# Step 2: find optimal parameters for PrIM
python prim_autotune.py

# Step 3: find optimal parameters for PrIM+search
python prim_search_autotune.py

# Step 4: find optimal parameter for SimplePIM
python simplepim_autotune.py

# Step 5: perform autotuning for ATiM
python atim_autotune.py
```

A.6 Evaluation and expected results

Once autotuning results are available, users can evaluate the execution times of tensor programs optimized using CPU-autotuned, PrIM/(E), PrIM+search, SimplePIM, and ATiM configurations. Users can also evaluate the impact of different levels of ATiM's PIM-aware optimizations as defined in Section 7.3.

```
# Evaluate tuned binaries/modules for tensor programs
python cpu_eval.py # CPU-autotuned
python prim_eval.py # PrIM/(E) and PrIM+search
python simplepim_eval.py # SimplePIM

python atim_eval.py # ATiM

# Evaluate ATiM's PIM-aware optimizations
python atim_branch_opt.py
```

Finally, the graphs in Figs. 9, 10, and 12 can be regenerated di-

rectly from the CSV files using the following command. The resulting graphs will be located in the `atim/evaluation/reproduced` directory.

```
python plot.py
```

A.7 Experiment customization

ATiM supports autotuning tensor programs across various workloads and tensor shapes. Users can run experiments on a specific workload with a custom tensor shape by supplying command-line arguments. For example, the following commands run autotuning and evaluation for an MMTV operation with a tensor shape of 256×512×256.

```
python atim_autotune.py \
    --workload=mmtv --m=256 --n=512 --k=256

python atim_eval.py \
    --workload=mmtv --m=256 --n=512 --k=256
```

Additionally, users can modify the following files to test different workload sizes or introduce new workloads.

- `evaluation/bench.py`: Define new workloads in TIR.
- `evaluation/tasks.py`: Configure workload sizes.
- `evaluation/workloads.py`: Register workloads to run experiments.